VASP-Flow

User Manual

Browser GUI · CLI Agents · SLURM HPC · High throughput · Diagnosis & repair

Keivan Esfarjani

v2.6 · October 2026 | https://github.com/KeivanS/VASP-Flow

Contents

I. Getting started 5

1. What VASP-Flow is for 5

2. Choose your route 6

3. Installing and configuring VASP-Flow 7
3.1. Requirements . . . 7
3.2. Install what your route needs . . . 7
3.3. Where each setting comes from . . . 8
3.4. default_paths — workstation configuration . . . 8
3.5. Configuring a cluster without the GUI . . . 9

II. Worked examples 9

4. Example 1 — Silicon in the browser: convergence tests, bands, DOS, ELF and bonding 10
4.1. Starting the GUI . . . 10
4.2. Step 1: the Setup tab . . . 10
4.3. Step 2: convergence tests (Workflow tab, phase 1) . . . 14
4.4. Step 3: production (Workflow tab, phase 2) . . . 14
4.5. Step 4: the Results tab . . . 14
4.6. Step 5: changing a tag by hand . . . 14
4.7. Viewing results of jobs run elsewhere . . . 16

5. Example 2 — GaAs from the command line on a workstation 18

6. Example 3 — GaAs on a SLURM cluster 21

7. Example 4 — High-throughput on a workstation: three materials from Materials Project 21

8. Example 5 — High-throughput on a cluster: three materials, with repair rounds 22

III. Reference: inputs, methods and options 26

9. The command-line agents: commands and the files they create 26
9.1. On a workstation: `vasp-agent.py` . . . 26
9.2. On a SLURM cluster: `vasp-agent-slurm.py` . . . 26
9.3. What the agent creates . . . 27

10. Making the plots: in the GUI or with `makeplots.sh` 27
10.1. On the command line (`makeplots.sh`) . . . 28

11. Describing a calculation in `instructions.txt`: every keyword 28
11.1. An example to start from . . . 28
11.2. All keywords . . . 29

12. Choosing functional, spin, SOC and GGA+U 30
12.1. Functionals . . . 30
12.2. Spin-orbit coupling . . . 31
12.3. Magnetic materials (collinear spin) . . . 31
12.4. GGA+U . . . 32

13. Convergence tests for ENCUT and k-points: syntax and outputs 32

14. Choosing the k-point mesh: densities, fixed meshes, DOS and LOBSTER 33

15. Setting INCAR and KPOINTS tags yourself 33
15.1. Raw INCAR passthrough . . . 34
15.2. Explicit KPOINTS blocks . . . 34
15.3. Command-line INCAR overrides (`-incar`) . . . 35
15.4. Constant-pressure relaxation . . . 35

16. Band structures, DOS projections and ELF profiles 35
16.1. Band-structure k-path . . . 35
16.2. Projected DOS . . . 36
16.3. ELF bond profiles . . . 36

17. LOBSTER bonding analysis (COHP, COBI, COOP) 37

18. Dielectric constant and Born charges (DFPT) 38
18.1. Example . . . 38

19. Wannier functions (Wannier90) 39

20. Phonons (phonopy) 39

21. Cores, nodes, KPAR/NCORE and walltimes 39
21.1. KPAR and NCORE . . . 39
21.2. Per-project resources (in `instructions.txt`) . . . 39
21.3. Walltime per step . . . 40

22. More `instructions.txt` examples: SOC, magnetism, phonons, pressure 40
22.1. MoS_2 monolayer with SOC . . . 40

22.2. Nickel: magnetism + GGA+U . . . 40
22.3. GaAs: phonons with NAC . . . 41
22.4. High-pressure relaxation with custom INCAR tags . . . 41
22.5. Bi_2Se_3: topological insulator . . . 42

23. High-throughput runs: the two drivers and their setup on a cluster 42
23.1. Step by step on the cluster (Rivanna) . . . 43

24. Scripts inside a high-throughput directory: submit, status, retune, collect 44

IV. Diagnosis and repair of failed runs 44

25. Repairing failed runs, round by round 44
25.1. The repair cycle in one command . . . 45
25.2. What a round tells you . . . 46
25.3. Where the failed run goes: round backups . . . 47
25.4. Setting parameters yourself . . . 47
25.5. Materials held for review . . . 48
25.6. Other runs: projects made with the agents . . . 49

26. Looking up one material or job quickly (`show_*.sh`) 49
26.1. A typical checking round . . . 50
26.2. `show_problems.sh`: what failed and what the fixes will do . . . 50
26.3. `skip_fixes.sh`: leave materials out . . . 50
26.4. `show_material_status.sh`: how far each step got . . . 50
26.5. `show_job_history.sh` and `show_job_log.sh`: what SLURM and the logs say . 51

27. Testing a fix on one material first 51

28. Auditing, collecting and cleaning up a high-throughput run 52

29. Troubleshooting: symptoms and remedies 54
29.1. Setting up and running . . . 54
29.2. High-throughput runs on a cluster . . . 55
29.3. Quick OUTCAR checks . . . 55

Appendices: how the inputs are chosen 56

A. How the inputs are generated 56
A.1. k-mesh formula . . . 56
A.2. KPAR and NCORE rule . . . 56
A.3. Eigensolver (ALGO) of each step . . . 57
A.4. LOBSTER k-mesh and memory . . . 57
A.5. LOBSTER charge spilling: what it is and how to lower it . . . 57
A.6. Relaxation → SCF hand-off . . . 59
A.7. Band-path detection . . . 59
A.8. Two-step ELF (`ELF: separate`) . . . 60
A.9. Convergence tests: test structure and parameter choice . . . 60
A.10. DFPT: fixed parallel and symmetry settings . . . 61
A.11. LOBSTER projected DOS vs. the VASP DOS . . . 61

B. How SLURM steps continue past the time limit **61**

C. High-throughput drivers: all options **62**
C.1. `ht-mp-scf.py` in detail 62
C.2. `ht_runs.py` in detail 63

D. How `vasp_diagnose.py` works **64**
D.1. Command line 64
D.2. The list file 65
D.3. Kinds of runs 66
D.4. When is a step OK? 66
D.5. Failure causes and their repairs 68
D.6. What is redone 69
D.7. Order of the resubmitted jobs 69
D.8. Output files 70
D.9. Limitations 70

E. Version history **71**

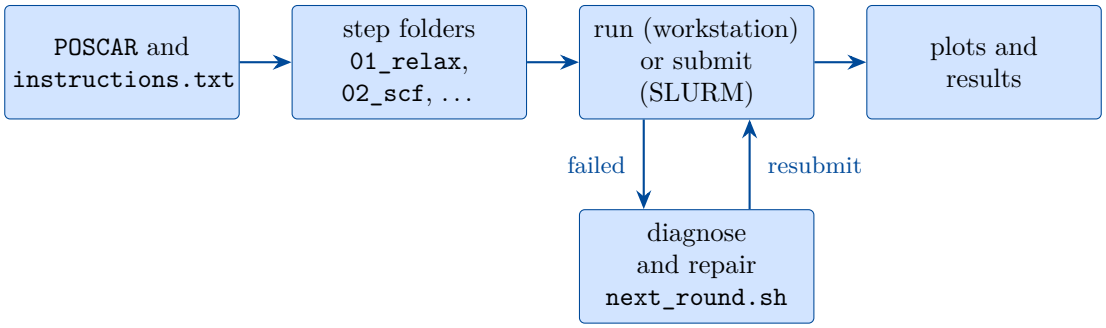
Part I.
Getting started

1. What VASP-Flow is for

VASP-Flow is a set of tools that do the routine work around VASP calculations, so that your time goes into the physics rather than into input files, job scripts and failed runs. You give it a structure (`POSCAR`) and a few lines such as "PBEsol, relaxation, SCF, band structure". The tools then

- **prepare and submit**: write every INCAR, KPOINTS, POTCAR and job script, and run the steps in order on your computer or submit them as chained SLURM jobs that continue by themselves past the walltime;
- **diagnose and repair**: read the output of every failed or unfinished step, name the cause (walltime, memory, SCF not converging, relaxation not finished, high LOBSTER spilling, . . .), say how certain that is, write the fix and resubmit only what has to be redone, round after round;
- **analyse**: plot bands, DOS, ELF, convergence and phonons, summarise Born charges and dielectric tensors, and collect LOBSTER bonding data from a whole batch into one table.

These matter most in a **high-throughput** run. For a few hundred materials, submitting the jobs, finding out which ones failed and why, fixing them and gathering the results take far more of your time than the calculations themselves; VASP-Flow turns each of these into one command. The same tools serve a single material, where they save you from writing the inputs by hand.



Tool	What it is for
`vasp-gui.py` (`make run`)	browser GUI: set up, run and plot one material
`vasp-agent.py`	the same from the terminal, on a workstation
`vasp-agent-slurm.py`	the same on a SLURM cluster: one job per step, chained
`ht_runs.py`, `ht-mp-scf.py`	high throughput: fetch a list of materials from Materials Project, set them all up and submit them
`vasp_diagnose.py`, `ht_tools/next_round.sh`	diagnosis and repair, one round per command
`ht_tools/show_*.sh`, `audit_run.sh`	where each material and job stands, and why
`makeplots.sh`, `collect_lobster.sh`, `lobster_postprocess.py`	plots, and the results of a batch gathered for a database

The GUI, the two agents and the high-throughput drivers share one input generator, so the same request gives the same files whichever way it was made. The methods covered are relaxation, SCF, bands, DOS, ELF, convergence tests, DFPT, phonons, Wannier90 and LOBSTER bonding, with R2SCAN (default), PBE, PBEsol, LDA or HSE06, spin, SOC and GGA+U.

2. Choose your route

I want to ...	Start here	You also learn
run one material in the browser	Example 1 (4)	convergence tests, bands and DOS, editing a tag
run one material from the terminal	Example 2 (5)	`instructions.txt`, INCAR blocks
run one material on a cluster	Example 3 (6)	SLURM jobs, rerunning one step
run many materials	Example 4 (7, workstation) or 5 (8, cluster)	lists from Materials Project; repair rounds and your own settings (5)
fix a failed batch	Section 25.1	reading the report, holding materials back

Before the example, install what your route needs (Section 3).

The rest of the manual: Part II — the worked examples, the only place with step-by-step commands; Part III — the reference: every field, keyword and option, and how to change it; Part IV — diagnosis and repair of failed runs; the appendices — how each value is chosen.

3. Installing and configuring VASP-Flow

3.1. Requirements

What	Needed for	How to get it
VASP 5.4+	everything: `vasp_std`, `vasp_ncl` (SOC), `vasp_gam` (optional)	your licence
POTCAR library	everything: PAW potentials in per-element folders (`PAW_PBE`; `PAW_LDA` for LDA)	with VASP
MPI	running VASP (`mpirun`, `mpiexec` or `srun`)	system / cluster module
Python $\geq$ 3.8, numpy, matplotlib	everything (agents, `makeplots.sh`)	`pip install numpy matplotlib`
Flask, sumo $\geq$ 2.0	the GUI and its band/DOS plots	`pip install flask sumo`
pymatgen, mp-api, spglib	the high-throughput drivers (structures from Materials Project, the Materials Project POTCAR choices); spglib also finds the band-structure k-path	`pip install pymatgen mp-api spglib`
Materials Project API key	the high-throughput drivers, for structures not downloaded before	free, https://materialsproject.org/api
LOBSTER	the LOBSTER bonding step (`08_lobster`)	free for academics, http://www.cohp.de
phonopy, wannier90.x	optional: phonons, Wannier functions	`conda install -c conda-forge phonopy wannier90`

Check the Python first. The system `python3` of a cluster is often too old (on Rivanna it is 3.6, without pymatgen): `ht_runs.py` then stops with `unexpected keyword argument 'capture_output'` or `No module named 'pymatgen'`. Make the environment of step 2 below once, and activate it at every login.

3.2. Install what your route needs

The same steps on your computer and on a cluster (there on the login node, which has the internet access the high-throughput drivers need). Lines marked for a route you do not take can be skipped.

```
# 1. The code (update later: git pull)
cd ~ && git clone https://github.com/KeivanS/VASP-Flow.git

# 2. A Python >= 3.8 environment, made once
python3 --version                # cluster: load a newer Python first
python3 -m venv ~/vaspenv
source ~/vaspenv/bin/activate    # at every login
pip install numpy matplotlib                # all routes
pip install flask sumo                      # GUI (Example 1)
pip install pymatgen mp-api spglib          # many materials (Ex. 4, 5)
# optional, in a conda env instead of the venv:
#   conda install -c conda-forge phonopy wannier90

# 3. Where VASP, MPI and the POTCARs are
```

```
cd ~/VASP-Flow
make setup && nano default_paths    # your computer (Ex. 1, 2, 4)
nano profiles/slurm.json            # a cluster (Ex. 3, 5)

# 4. Many materials: Materials Project key (or in ~/.bashrc)
export MP_API_KEY=your_key       # free: materialsproject.org/api

# 5. Check
make check
```

Is this computer ready? Step 5 runs `check_setup.py` (also `python3 check_setup.py` without make). It checks Python and its packages, the POTCAR library, the VASP and MPI programs and, on a cluster, `profiles/slurm.json` and the high-throughput packages, and prints for each problem the command that fixes it. Without arguments it checks what fits the computer (a cluster where `sbatch` exists, else a workstation); `make check FOR="slurm ht"` names the routes yourself (`gui`, `local`, `slurm`, `ht`). **Ready.** at the end means you can start the example. Run it again after every change of the set-up.

3.3. Where each setting comes from

One table for all of VASP-Flow; the first source that has a value wins.

Setting	Tool	Source, highest first
programs (VASP, MPI, LOBSTER), POTCAR library	GUI	*System Setup* page > `default_paths` > shell variables
	`vasp-agent.py`	`profiles/workstation.json` (with `--profile workstation`) > shell variables
	`vasp-agent-slurm.py`	`profiles/slurm.json` > shell variables
	`ht_runs.py` jobs	`env.sh` of the run folder (Section 24)
job size, walltime	agents	`instructions.txt` (`<STEP>_WALLTIME` > `WALLTIME`) > per-step defaults > `profiles/slurm.json`
	high throughput	`set_params.sh` (one material) > `env.sh` (`TIME_*`)
INCAR, KPOINTS	agents, GUI	blocks in `instructions.txt` > its keywords > the automatic choice
	repair rounds	`set_params.sh` > the automatic fix

Shell variables are `VASP_STD`, `VASP_POTCAR_DIR`, ...; `default_paths` reaches the agents only through `make` or when its values are exported in the shell.

3.4. default_paths — workstation configuration

The file `default_paths` (Makefile key=value syntax, made by `make setup` from `default_paths.example`) is read by `make run`; its variables become the GUI's initial settings (precedence: Section 3.3).

```
PYTHON           = python3
VASP_STD         = /path/to/vasp_std
VASP_NCL         = /path/to/vasp_ncl
VASP_GAM         = /path/to/vasp_gam       # optional
VASP_POTCAR_DIR  = /path/to/potpaw_PBE
VASP_POTCAR_DIR_LDA = /path/to/potpaw_LDA # LDA only
```

```
MPI_LAUNCH      = mpirun -np                # or: srun
MPI_NP          = 16
WANNIER90_X     = wannier90.x
LOBSTER_X       = lobster                   # or full path
VISUALIZER_X    = ~/bin/jmol                # Jmol, VESTA, ...
```

Two POTCAR libraries. Every functional uses the PBE potentials of `VASP_POTCAR_DIR` except LDA, which needs the LDA potentials of `VASP_POTCAR_DIR_LDA` (GUI System Setup: *LDA POTCAR library*). Left empty, it is the PBE folder with PBE replaced by LDA in its name (`PAW_PBE` → `PAW_LDA`, `potpaw_PBE.54` → `potpaw_LDA.54`); if that folder does not exist an LDA run stops with an error instead of using PBE potentials. The agents print which library they used (`POTCAR library: ...`); for the high-throughput driver the cluster path is `--hpc-potcar-dir-lda`.

3.5. Configuring a cluster without the GUI

On a cluster without the GUI, the SLURM agent takes the VASP binaries, MPI command, modules and default job size from `profiles/slurm.json`; edit it once after cloning:

```
{
  "vasp_std":   "/path/to/vasp_std",
  "vasp_ncl":   "/path/to/vasp_ncl",
  "vasp_gam":   "/path/to/vasp_gam",
  "wannier90_x": "wannier90.x",
  "lobster_x":  "/path/to/lobster",
  "mpi_cmd":    "srun",
  "mpi_np":     128,
  "modules":    ["intel/2023", "vasp/6.4"],
  "slurm": {
    "partition": "standard",
    "nodes":      2,
    "ntasks_per_node": 64,
    "time":       "12:00:00",
    "account":    "your-project"
  }
}
```

The `modules` list is prepended as `module load ...` lines in every generated `run.sh`. Alternatively, all path keys can be set as environment variables in your `.bashrc` (`VASP_STD`, `LOBSTER_X`, etc.) and the profile values left as empty strings to inherit them.
Save Settings in the GUI rewrites `profiles/slurm.json`; without the GUI, edit the file and leave it in place. Per-project values (nodes, walltimes, ...) go into `instructions.txt` (Section 21).

Part II. Worked examples

Five complete recipes, from one material in the browser to a high-throughput run on a cluster that needs a repair round (choose one with the table of Section 2). Each starts with a short card — where it runs, how long it takes, what success looks like, what to do next — and assumes that `make check` passed. You can type the commands as they are.

4. Example 1 — Silicon in the browser: convergence tests, bands, DOS, ELF and bonding

At a glance	
Runs on	your computer, with VASP, MPI and LOBSTER (`make check FOR="gui local"`)
Takes	about half an hour on 8 cores, most of it the convergence tests
Success	the Results tab shows the band structure with the gap of step 5, the DOS, the ELF and the COHP of the Si–Si bond
Next	Example 2 (the same without the browser)

This example is also the reference of the browser GUI: every field, button and panel is described where the example first uses it.

4.1. Starting the GUI

In the VASP-Flow folder run `make run` and open http://localhost:5001. On a SLURM cluster, start it on the login node and open it on your laptop through an SSH tunnel:

```
ssh -L 5001:localhost:5001 username@cluster      # on your laptop
cd ~/VASP-Flow && make run                        # on the login node
# then open http://localhost:5001 in the laptop's browser
```

The GUI has three tabs: **Setup** (describe the calculation, then *Generate Workflow*), **Workflow** (run the steps; the VASP output streams into the page) and **Results** (plots and the OUTCAR summary). The bar under the tabs returns to an earlier project: choose it under *Resume project*; → **Workflow** and **Results** jump to those tabs, **Edit & Regenerate** reopens Setup with every field filled in. The first time, open *System Setup* at the top of the Setup tab, enter the paths of VASP, the POTCARs and MPI (they are stored in `settings.json`) and click *Save Settings*.

4.2. Step 1: the Setup tab

Project and structure. Enter the project name `Si_example`, 8 MPI cores, and paste the primitive cell of silicon into the POSCAR box (Figure 1):

```
Si
1.0
   0.000    2.734    2.734
   2.734    0.000    2.734
   2.734    2.734    0.000
Si
2
Direct
   0.00  0.00  0.00
   0.25  0.25  0.25
```

Field	Meaning
Project name	the folder created in the projects folder; a new name starts a new project without overwriting the old one
Execution profile	*Workstation* runs VASP on this computer; *SLURM HPC* writes `sbatch` scripts (set `profiles/slurm.json` first)
MPI cores	cores per calculation
POSCAR	paste the structure or *Load file*; *Visualize* opens it in the viewer of System Setup (Jmol, VESTA, OVITO, . . .)
POTCAR directory	blank = the library of System Setup; an element with several variants (`Ga`, `Ga_d`) gets a chooser

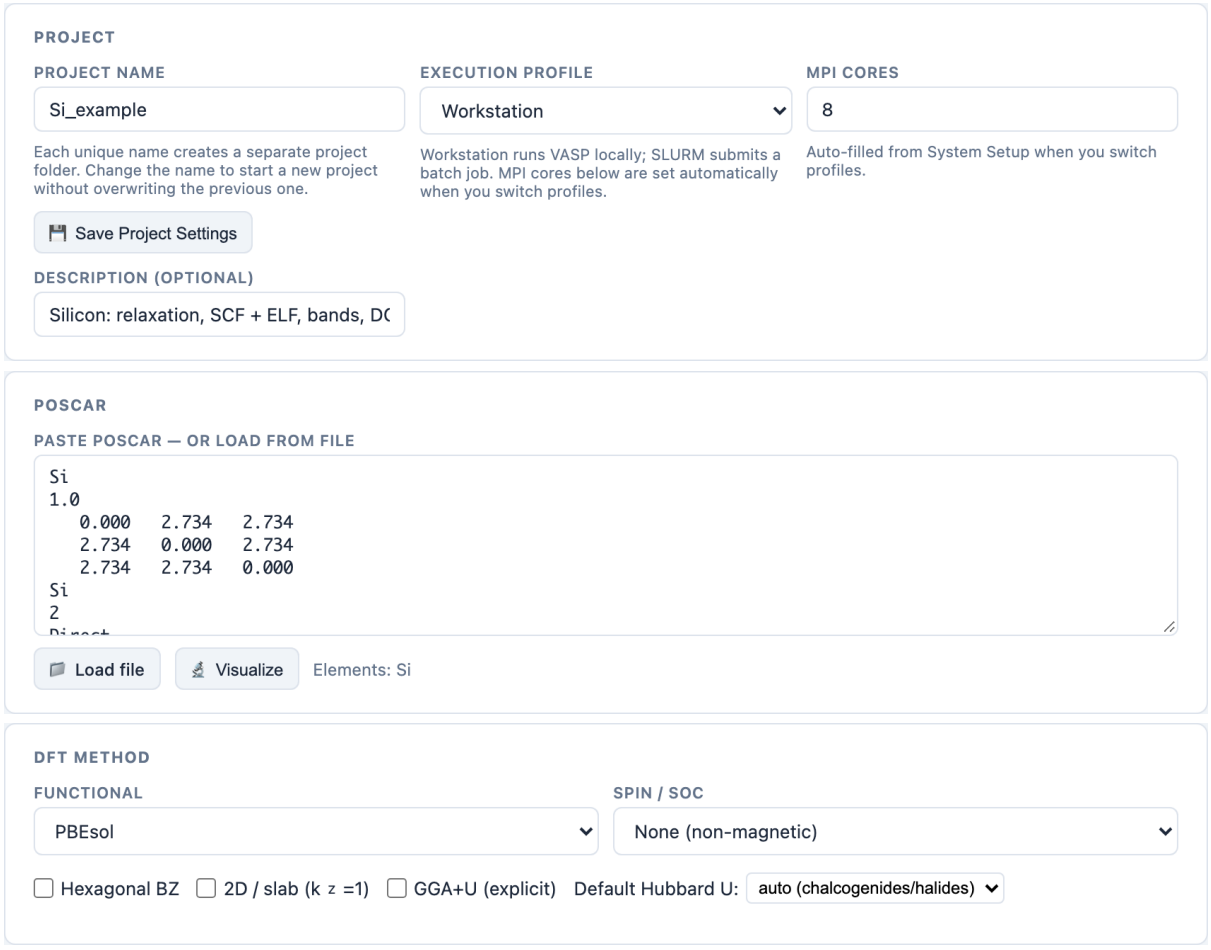


Figure 1: Setup tab, top: project, structure and method for silicon.

Method. Choose **PBEsol** (the default R2SCAN is better but slower) and **Spin / SOC**: None. *Hexagonal BZ* stays unticked: the GUI ticks it itself when the pasted cell is hexagonal, and silicon is cubic.

Choice	Meaning
R2SCAN	**default**; meta-GGA, better structures, energies and gaps than the GGAs at 1.5–3× the cost
PBE, PBEsol	GGAs; PBE is what the high-throughput drivers use, PBEsol gives good lattice constants
LDA	`GGA = CA`, with the LDA POTCARs (`VASP_POTCAR_DIR_LDA`, §3.4)
HSE06, RVV10, AM05	hybrid (accurate gaps, much slower); van der Waals for layered materials; rarely needed
Spin: None	non-magnetic
Collinear spin	`ISPIN = 2`; shows the *Magnetic moment / atom* field (Section 12.3)
SOC, non-magnetic	`LSORBIT`, `MAGMOM = 3N*0`, `vasp_ncl`; for non-magnetic materials with heavy elements (Bi_2Se_3, MoS_2)
SOC ∥ z/x/y	the same with initial moments along z, x or y; magnetic materials only
Hexagonal BZ	ticked automatically for a hexagonal cell; matters only for a k-path typed by hand with labels spglib does not know (the M point of the generic table)
2D / slab	$k_z = 1$: one k-point across the vacuum
GGA+U (explicit)	your own U per element and orbital; *Default Hubbard U* chooses the tabulated values (Section 12.4)

Convergence tests and tasks. Under *Calculation parameters* keep *Run convergence tests first* and enter the k-point meshes `4x4x4, 6x6x6, 8x8x8, 10x10x10, 12x12x12` and the ENCUT range `250-500`. Under *Tasks* keep Structure relaxation, SCF with ELF, Band structure and DOS, add a projection row (element `Si`, orbitals `s` and `p`), and tick *LOBSTER bonding analysis* (Figure 2). The *K-path* field is already filled: as soon as the POSCAR was pasted, the GUI asked spglib for the standard path of the cell, here the FCC path `G-X-W-K-G-L-U-W-L-K|U-X` (Section 16.1; `|` starts a new segment). Keep it, or type your own, e.g. `G-X-W-K-G-L`; a path you typed is kept when you paste another POSCAR.

Task	Folder	What it does
Structure relaxation	`01_relax`	cell and ions (ISIF, NSW, EDIFFG in the task)
SCF (+ ELF)	`02_scf`	charge density; needed by every later step; ELF writes ELFCAR
Band structure	`03_bands`	along the k-path; energy window of the plot
DOS	`04_dos`	twice the SCF mesh; one projection row per element
LOBSTER	`08_lobster`	symmetry-off NSCF, then COHP/COBI/COOP (Section 17)
DFPT	`06_dfpt`	dielectric constant and Born charges (Section 18)
Phonons	`07_phonons`	phonopy supercells (Section 20)
Wannier90	`05_wannier`	NSCF and `wannier90.win` (Section 19)

Click **Generate Workflow** at the bottom of the tab. The project folder `Si_example/` is created with one sub-folder per step; its layout is that of Example 2 (Figure 7).

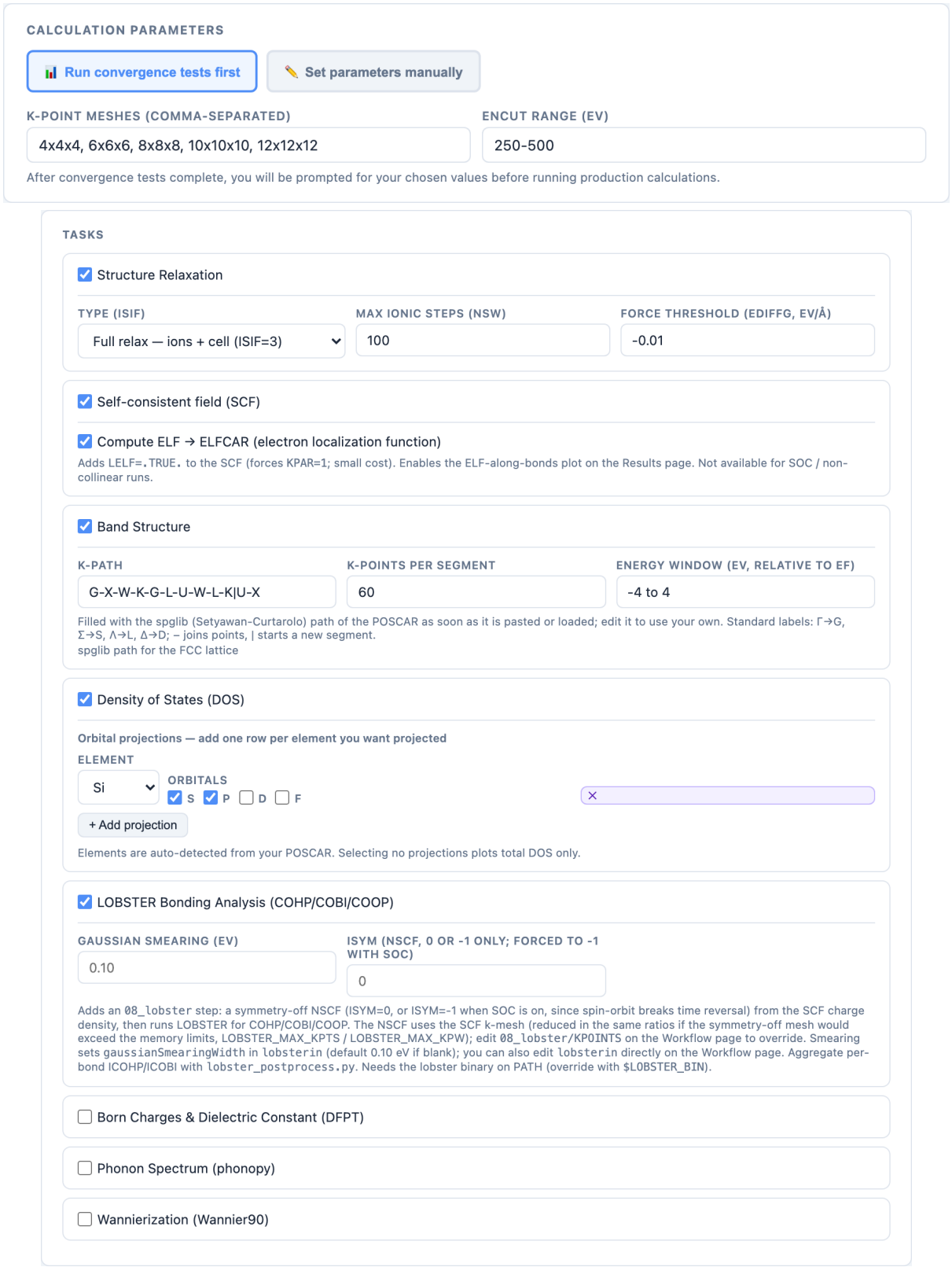


Figure 2: Setup tab, lower part: convergence tests and the tasks of this example.

4.3. Step 2: convergence tests (Workflow tab, phase 1)

On the **Workflow** tab every step has **Run**, **Edit files** (its INCAR, KPOINTS and POSCAR in the browser; a `.bak` copy is kept) and a status badge; **Run All Steps** runs them in order and stops at the first failure. Run *Convergence tests*. Each ENCUT and each k-mesh is one short SCF; the plots of energy, pressure and force against ENCUT and against the k-mesh appear below the button. When they are done, the fields under *Review results then set production parameters* hold the automatic choice: the smallest ENCUT and k-mesh after which the force and the pressure no longer change (by more than 5 meV/Å and 0.5 kbar). For silicon this is typically ENCUT ≈ 300–350 eV and an 8×8×8 to 10×10×10 mesh. You may type other values.

4.4. Step 3: production (Workflow tab, phase 2)

Click **Run Phase 2 with these parameters**. ENCUT is written into every INCAR, the k-meshes are regenerated (the DOS gets twice the SCF mesh) and relax → SCF → bands → DOS → LOBSTER run one after the other. Each step takes its structure and charge density from the one before.

4.5. Step 4: the Results tab

The plots are drawn from the outputs when you open the tab — nothing has to be run first — and saved in `Si_example/plots/` (plots that `makeplots.sh` already made are shown as they are). Each group has a bar *Energy window (eV): min to max*: **Zoom** redraws it for that range, **Reset** returns to the default, and the **PDF** buttons give the vector files. The default band window is the *Energy window* of the band task.

Panel	What it shows
Calculation summary	per step: final energy, Fermi level, moment, converged; *OUTCAR* shows the energy, Fermi level, moment and forces of that step
Band structure	$E_F = 0$; spin up blue, spin down red dashed; fat bands weighted by orbital (needs PROCAR); sumo, with a built-in plotter if sumo fails
Density of states	total, and projected on the orbitals of the projection rows (spin down mirrored)
ELF	along the nearest-neighbour bonds and in the plane of an atom and its neighbours (needs ELFCAR; not for SOC)
LOBSTER	−COHP, COBI, COOP per bond type and their integrals; the LOBSTER projected DOS against the VASP total (the gap between them is the charge spilling, Appendix A.5)
Born charges, phonons	after `06_dfpt` (*Extract from OUTCAR*) and `07_phonons`

Silicon should show an indirect gap of about 0.5–0.6 eV (PBEsol underestimates the experimental 1.1 eV), a DOS dominated by Si-*p* at the valence-band top, an ELF of about 0.95 in the middle of the Si–Si bond, and a bonding −COHP below E_F (ICOHP about −4.5 eV per Si–Si bond, COBI about 0.9).

4.6. Step 5: changing a tag by hand

- **One file, now**: on the Workflow tab, **Edit files** next to a step opens its INCAR, KPOINTS and POSCAR. Change e.g. `SIGMA = 0.05` to `SIGMA = 0.02` in `03_bands/INCAR`, save, and run that step again.
- **In the project, for good**: **Edit & Regenerate** reopens Setup. Under *Advanced — explicit INCAR / KPOINTS* (Figure 6) choose a step and type tags, one `TAG = value`

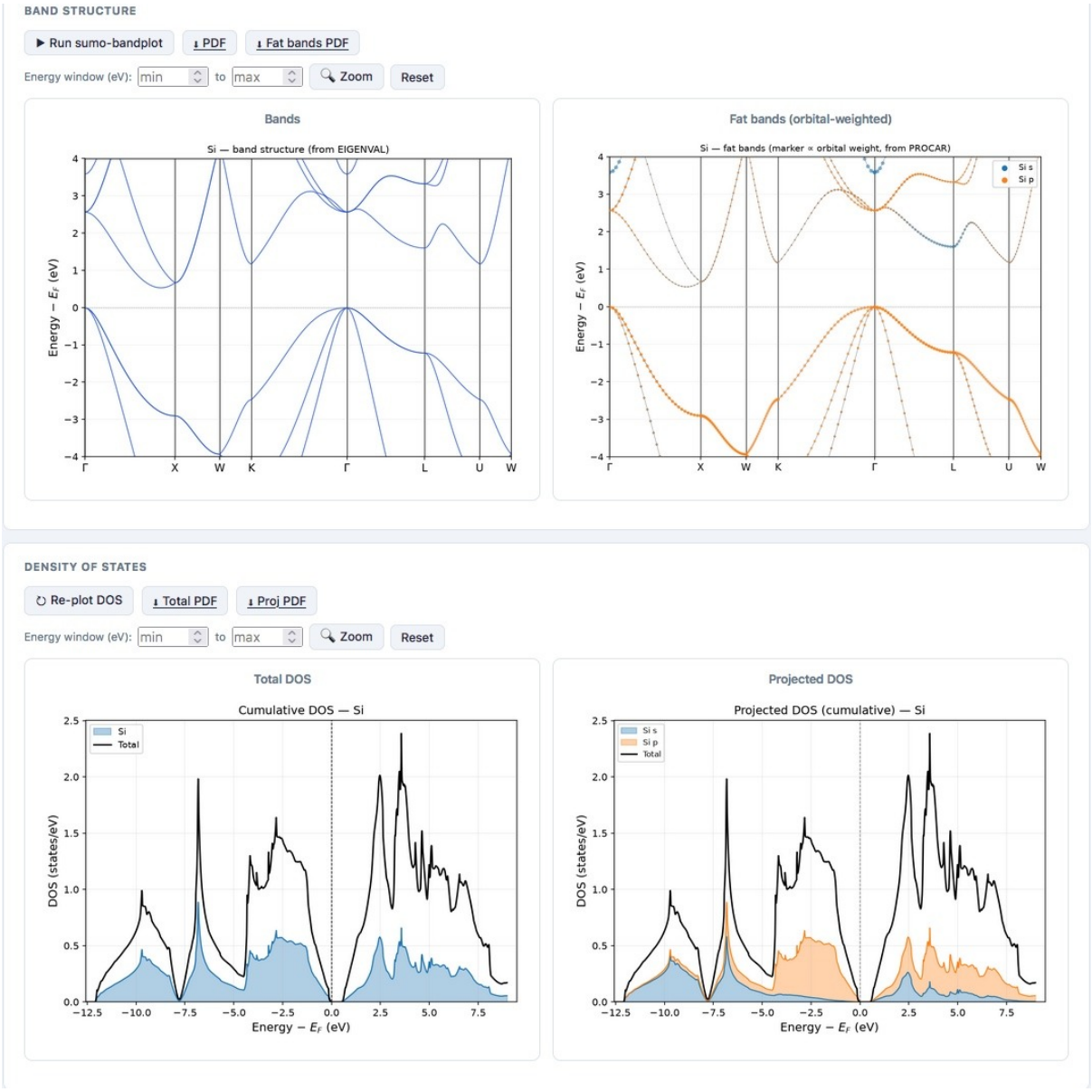


Figure 3: Results tab for silicon: the band structure (plain and weighted by the Si-s and Si-p character) and the total and projected DOS. The indirect gap is about 0.55 eV, with the conduction-band minimum on the Γ–X line; the top of the valence band is Si-p. (This run is R2SCAN at the PBEsol lattice constant, slightly too small for R2SCAN; at its own lattice constant R2SCAN gives 0.6 eV.)

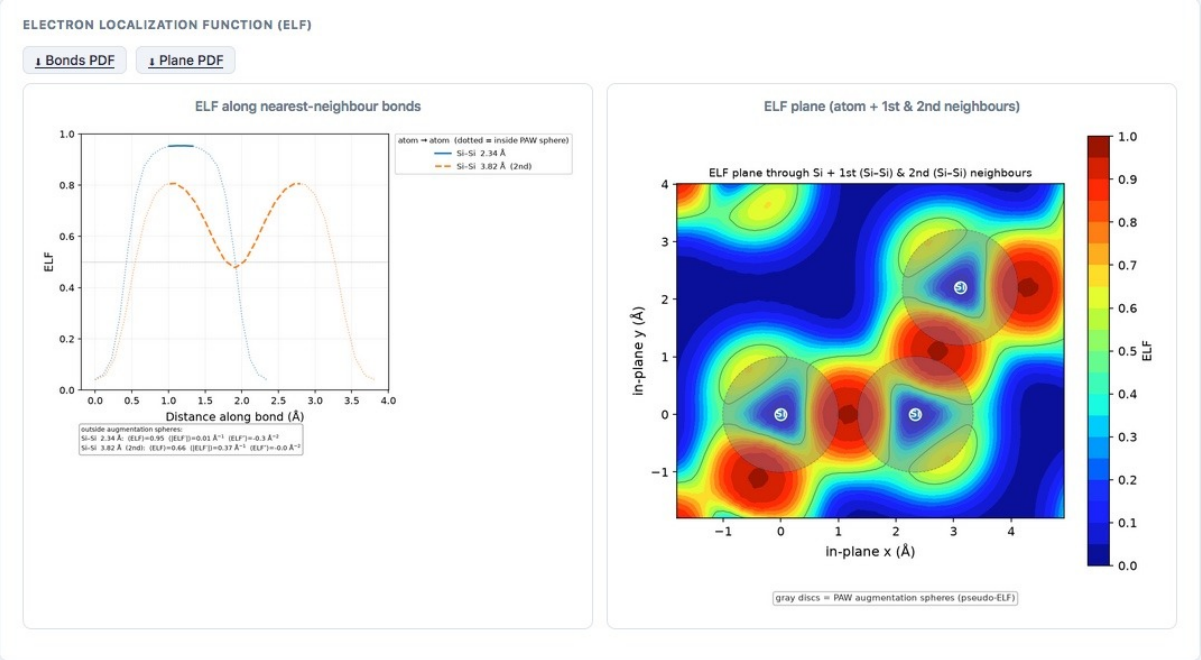


Figure 4: Results tab for silicon: the ELF along the first- and second-neighbour Si–Si lines (about 0.95 in the middle of the bond; dotted inside the PAW spheres, where it is not meaningful), and in the plane through an atom and its neighbours.

per line, or a whole KPOINTS file; then **Generate Workflow** again. These override every automatic choice (Section 15).

4.7. Viewing results of jobs run elsewhere

The Results tab also shows calculations the GUI did not set up or run: a cluster job, a whole high-throughput run, or the tarball of `collect_results.sh`. Type the path in **Open results folder** on the bar under the tabs and press **Open**:

Path	What is opened
one calculation, e.g. `.../materials/mp-8`	that project (listed as `mp-8`)
a high-throughput folder, e.g. `.../HT_example5`	every `materials/<id>` (listed as `HT_example5~mp-8`, ...)
an unpacked `screen_results.tar.gz` (`.../results`)	every `results/<id>`

The opened calculations join the project list (marked *opened*) and are remembered in `settings.json`. Plots missing from `plots/` are drawn from the raw outputs — bands from `03_bands/EIGENVAL`, COHP/COBI/COOP and the LOBSTER DOS from `08_lobster/*.lobster`, ELF from `02_scf/ELFCAR` — and saved there. Nothing is submitted or rerun. A step counts as done when its OUTCAR shows VASP finished (a relaxation must also reach the ionic criterion). **Forget opened** removes them from the list without touching any file. `collect_results.sh` keeps this layout with trimmed OUTCARs, so its tarball opens directly; set `FULL_OUTCAR=1` to keep the complete OUTCARs.

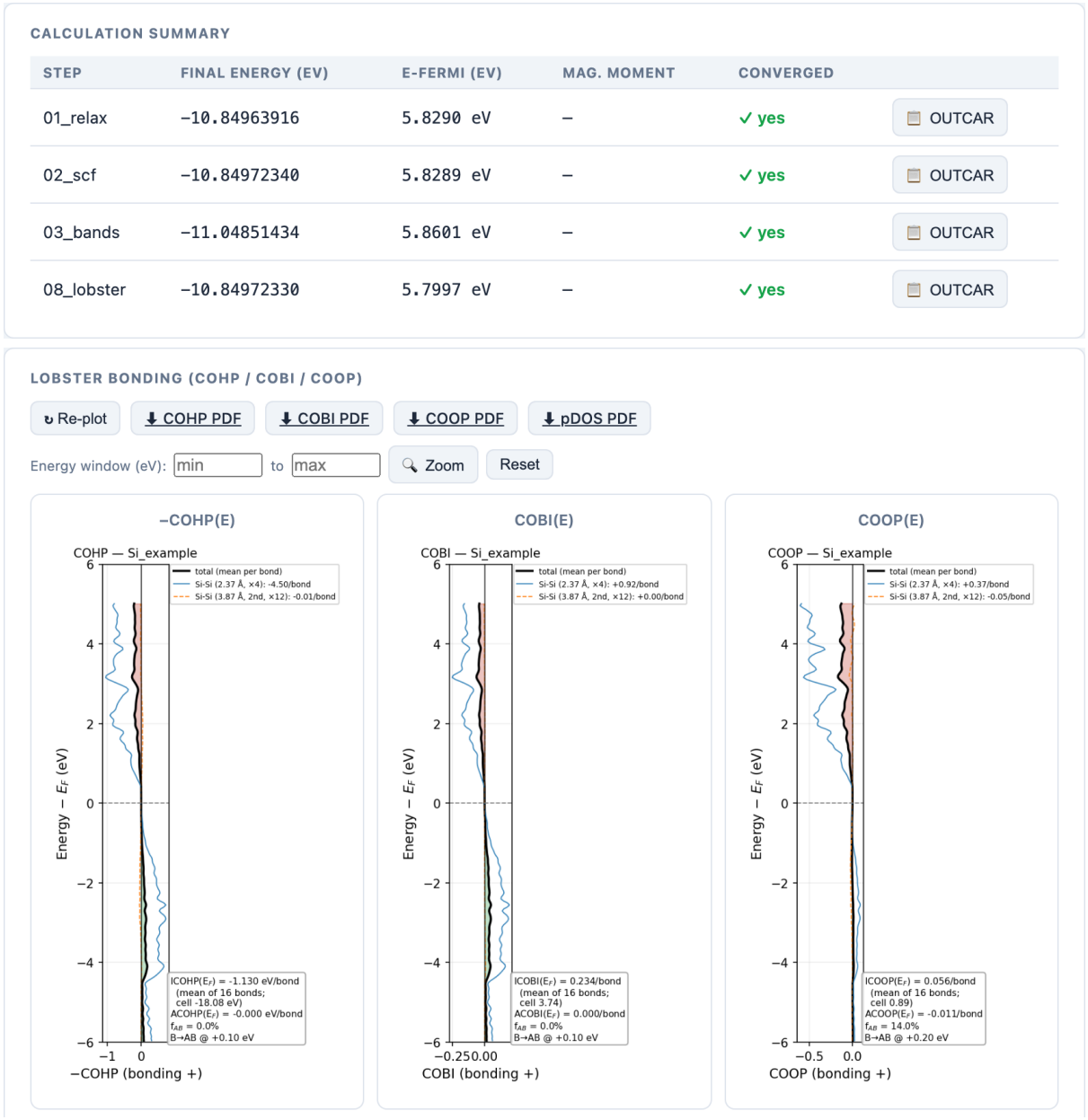


Figure 5: Results tab for silicon (the Si run mp-149 of the high-throughput database, PBE, so there is no `04_dos`): the summary of every step, and LOBSTER −COHP, COBI and COOP for the first (2.37 Å, ICOHP = −4.5 eV per bond) and second neighbours, with their integrals up to E_F. Below them the tab also shows the LOBSTER projected DOS (needs `DOSCAR.lobster`, not kept in that run).

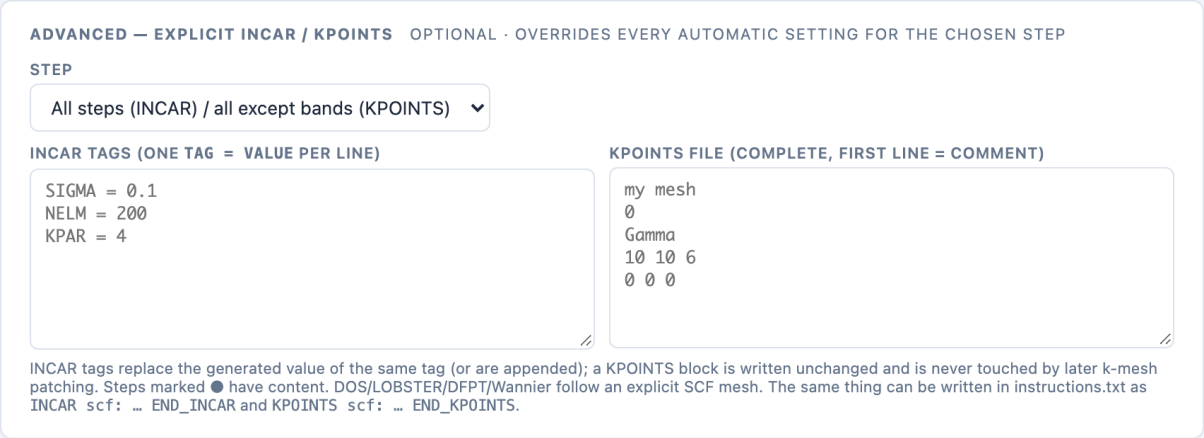


Figure 6: Setup tab, bottom: your own INCAR tags and KPOINTS file for one step or all.

5. Example 2 — GaAs from the command line on a workstation

At a glance	
Runs on	a workstation (`make check FOR=local`)
Takes	about an hour on 8 cores
Success	`plots/GaAs_example_band.png` shows the direct gap of GaAs at Γ
Next	Example 3 (the same on a cluster); all keywords: Section 11.2

The same steps without the browser, with `vasp-agent.py`. Work in a folder of your own, outside the VASP-Flow folder.

1. Two input files. `POSCAR` (primitive zinc-blende GaAs) and `instructions.txt`:

```
GaAs
1.0
   0.000    2.827    2.827
   2.827    0.000    2.827
   2.827    2.827    0.000
Ga As
1 1
Direct
   0.00 0.00 0.00
   0.25 0.25 0.25
```

```
Project: GaAs_example
Methods: PBEsol functional
Tasks: structure relaxation
       SCF calculation
       band structure
       DOS (projected on Ga-p and As-p)
Convergence: Test k-points 4x4x4, 6x6x6, 8x8x8, 10x10x10
             Test ENCUT 250:50:500
MPI: 8

# my own tags: tighter SCF, smaller smearing for the bands
INCAR scf:
   EDIFF = 1E-7
END_INCAR
INCAR bands:
```

```
   SIGMA = 0.02
END_INCAR
```

`instructions.txt` is plain English with a few keywords; all of them are listed in Section 11.2. The `INCAR scf: ... END_INCAR` blocks put your own tags into one step's INCAR and override the generated values.

2. Generate the inputs. Make a folder, move the two files into it (here they were saved in `~/Downloads`) and run the agent there:

```
mkdir -p ~/calc
mv ~/Downloads/POSCAR ~/Downloads/instructions.txt ~/calc/
cd ~/calc
python3 ~/VASP-Flow/vasp-agent.py
```

Without arguments the agent reads `instructions.txt` and `POSCAR` of the current folder; other names are given with `-i my_inst.txt -s my.vasp`. It creates `GaAs_example/` with `00_convergence/`, `01_relax/`, `02_scf/`, `03_bands/`, `04_dos/` (each with INCAR, KPOINTS, POTCAR and `run.sh`) and the scripts `run_convergence.sh`, `run_calculations.sh`, `makeplots_convergence.sh` and `makeplots.sh`. Look at the INCARs: they are complete and commented, and `02_scf/INCAR` ends with your `EDIFF`. The whole folder is drawn in Figure 7.

3. Phase 1: convergence tests. Run the tests, plot them and look at the automatic choice:

```
cd GaAs_example
./run_convergence.sh          # one short SCF per ENCUT, mesh
./makeplots_convergence.sh    # plots in 00_convergence/
cat 00_convergence/converged_params.txt    # the choice
```

4. Phase 2: production. Then run the calculations and the plots:

```
./run_calculations.sh    # asks ENCUT, k-mesh (Enter = automatic)
./makeplots.sh           # bands, DOS, OUTCAR summary in plots/
```

`run_calculations.sh` writes the chosen ENCUT and mesh into every step and runs relax → SCF → bands → DOS. `makeplots.sh` writes `plots/GaAs_example_band.png`, `..._dos.png` and their PDFs; to view the project in the browser instead, start the GUI and select it (Section 10). Without a `Convergence:` line the agent writes a single `run_all.sh` instead.

5. Change inputs by hand. Three ways:

- Before a step runs, its files are yours: edit `03_bands/INCAR` and run that step alone with `cd 03_bands && bash run.sh`.
- To change the project for good, edit `instructions.txt` and run the agent again; a fixed mesh goes in a `KPOINTS scf: ... END_KPOINTS` block (Section 15.2).
- One-off tags on the command line: `vasp-agent.py -incar "scf:NELM=200"` (Section 15.3).

The folder of one calculation (`vasp-agent.py`, Example 2)

folder written at set-up written by the runs written by the repairs

`~/calc/GaAs_example/`	made by `vasp-agent.py` in `~/calc`
├ `instructions.txt, POSCAR, POTCAR`	your input; POTCAR built from the library
├ `run_convergence.sh`	phase 1: the ENCUT and k-mesh tests
├ `makeplots_convergence.sh`	their plots and the automatic choice
├ `run_calculations.sh`	phase 2: asks ENCUT and mesh, runs the steps
├ `makeplots.sh, dos_proj.json`	the plots; the DOS projections asked for
├ `00_convergence/`	ENCUT and k-mesh tests
│ ├ `choose_params.py, plot_convergence.py`	
│ ├ `encut/`	INCAR KPOINTS POSCAR POTCAR run.sh
│ │ └ `encut_250/ ... encut_500/`	one SCF per ENCUT
│ ├ `kpoints/`	INCAR POSCAR POTCAR run.sh
│ │ └ `4x4x4/ ... 10x10x10/`	one SCF per mesh
│ ├ `converged_params.txt`	the automatic ENCUT and mesh
│ └ `encut_*.png, kpoints_*.png`	energy, pressure, force
├ `01_relax/`	cell and ions
│ ├ `INCAR KPOINTS POSCAR POTCAR run.sh`	
│ └ `OUTCAR OSZICAR CONTCAR CHGCAR WAVECAR`	
├ `02_scf/`	charge density
│ ├ `INCAR KPOINTS POSCAR POTCAR run.sh`	
│ ├ `copy_from_relax.sh`	takes CONTCAR and CHGCAR
│ └ `OUTCAR CHGCAR DOSCAR vasprun.xml`	
├ `03_bands/`	band structure
│ ├ `INCAR KPOINTS POTCAR run.sh copy_from_scf.sh`	KPOINTS: the k-path
│ └ `OUTCAR EIGENVAL PROCAR vasprun.xml`	
├ `04_dos/`	density of states, twice the SCF mesh
│ ├ `INCAR KPOINTS POTCAR run.sh copy_from_scf.sh`	
│ └ `OUTCAR DOSCAR vasprun.xml`	
└ `plots/`	plotting scripts
└ `GaAs_example_band.png, GaAs_example_dos.png`	and their PDFs

Figure 7: The project folder of Example 2. Every step folder is an ordinary VASP folder: edit its INCAR and rerun it with its `run.sh`. The GUI (Example 1) makes the same folder. On a cluster (`vasp-agent-slurm.py`, Example 3) the layout is the same, with `submit_convergence.sh` and `submit_calculations.sh` in place of the `run_*.sh` scripts; each `run.sh` is then an `sbatch` script and writes its log `slurm-<jobid>.out` into its step folder. Without a `Convergence:` line there is no `00_convergence/`, and one `run_all.sh` (`submit_all.sh`) runs every step.

6. Example 3 — GaAs on a SLURM cluster

At a glance	
Runs on	the login node of a SLURM cluster (`make check FOR=slurm`)
Takes	about an hour of compute time, plus the time in the queue
Success	the queue is empty and `vasp_diagnose.py` (step 3) reports every step `ok`
Next	many materials: Example 5; nodes and walltimes: Section 21

Exactly Example 2, with `vasp-agent-slurm.py`: every step becomes a SLURM job.

1. Inputs. The same `POSCAR` and `instructions.txt`, with the job size instead of `MPI: 8`:

```
NODES: 1
NTASKS_PER_NODE: 40
RELAX_WALLTIME: 02:00:00    # optional; default per step
```

2. Generate, then run the two phases. On the login node, move the two files (here copied to your home folder, e.g. with `scp`) into a folder on scratch and run there:

```
mkdir -p /scratch/$USER/calc
mv ~/POSCAR ~/instructions.txt /scratch/$USER/calc/
cd /scratch/$USER/calc
python3 ~/VASP-Flow/vasp-agent-slurm.py
cd GaAs_example
./submit_convergence.sh       # phase 1: the test jobs
squeue -u $USER               # wait until they are gone
./makeplots_convergence.sh
./submit_calculations.sh      # phase 2: relax->scf->bands->dos
./makeplots.sh                # after the jobs have finished
```

The production jobs are chained (`-dependency=afterok`): the SCF starts only when the relaxation ended well. Run `makeplots.sh` on the login node after the jobs, or copy the project to your computer and open it in the GUI (Section 10). Each step has its own walltime, and a step that reaches it is stopped cleanly and continued in a new job automatically (Section 21.3).

3. If a step failed. Find out why and repair it with the diagnosis (Part IV); for one project the list has one line:

```
echo GaAs_example > mylist.txt
python3 ~/VASP-Flow/vasp_diagnose.py mylist.txt     # ->
    diagnosis_round1/report.txt
```

To rerun one step by hand after editing its INCAR: `cd 03_bands && sbatch run.sh`.

7. Example 4 — High-throughput on a workstation: three materials from Materials Project

`ht-mp-scf.py` takes a list of Materials Project IDs, downloads each primitive cell and runs relax → SCF (+ ELF) → LOBSTER for each with `vasp-agent.py`.

At a glance	
Runs on	a workstation with internet access (`make check FOR="local ht"`)
Takes	a few hours: the three materials run one after another
Success	`diagnosis_round1/report.txt` lists all three as `ok`
Next	the same on a cluster with repair rounds: Example 5; both drivers: Section 23

```
source ~/vaspenv/bin/activate          # made once at installation
export MP_API_KEY=your_key
export VASP_POTCAR_DIR=/path/to/potpaw_PBE
mkdir -p ~/ht_small && cd ~/ht_small
printf 'mp-149\nmp-2534\nmp-22862\n' > my_list.txt    # Si, GaAs,
    NaCl
python3 ~/VASP-Flow/ht-mp-scf.py -l my_list.txt --agent local --mpi 8
./runall.sh                # builds and runs the three materials, one
    after another
```

Each material gets a project folder like those of Example 2 (`mp-149/01_relax`, `02_scf`, `08_lobster`, ...). The settings written for each material are in `_ht_inputs/<id>/instructions.txt`; edit one and run `runall.sh` again to change that material. Figure 8 shows the folder.

Check the runs. When `runall.sh` has finished:

```
python3 ~/VASP-Flow/vasp_diagnose.py my_list.txt
less diagnosis_round1/report.txt
```

Every material should be `ok`. If one is not, its repair script is in `diagnosis_round1/fix/`; for a workstation it runs VASP directly, so start it where VASP may run: `bash diagnosis_round1/fix/ht_small__mp-2534.sh`.

8. Example 5 — High-throughput on a cluster: three materials, with repair rounds

At a glance	
Runs on	the login node of a SLURM cluster (`make check FOR="slurm ht"`)
Takes	one or two days, mostly in the queue, with one or two repair rounds
Success	`next_round.sh` says `Nothing to submit.` and `audit_run.sh` shows every material ok
Next	your own list: Section 23.1; reading the report: Part IV

`ht_runs.py` is the cluster driver for a list of materials: relax → SCF → bands → LOBSTER, one SLURM job per step, every setting chosen per material. This example uses three materials picked so that at least one needs a second round:

Material	Why it is in the list
`mp-149` (Si)	an easy reference: finishes in minutes
`mp-972937` ($Sc_2Al_2C_2O_2$)	a dense LOBSTER mesh; with the settings of step 2 its LOBSTER step runs out of memory and is repaired in round 1
`mp-1080459`	its relaxation did not converge in the two passes of the first job in our runs, so round 1 continues it from its last structure

1. Generate the run. On the login node (it downloads from Materials Project):

```
cd ~/VASP-Flow && git pull
source ~/vaspenv/bin/activate          # made once at installation
    (pymatgen, mp-api, ...)
export MP_API_KEY=your_key
printf 'mp-149\nmp-972937\nmp-1080459\n' > lists/example5.txt
python3 ht_runs.py --list lists/example5.txt --single-node \
    --out /scratch/$USER/VASP-Flow/HT_example5
```

Figure 9 shows the folder it makes. Every material has the same steps, and each step folder is an ordinary VASP folder: you can look into it, edit its INCAR and rerun it.

The folder of a high-throughput run (`ht-mp-scf.py`, Example 4)

folder written at set-up written by the runs written by the repairs

`~/ht_small/`	where `ht-mp-scf.py` ran
`my_list.txt`	your mp-IDs
`runall.sh`	builds and runs the materials in turn
`_ht_inputs/`	what the agent is given, per material
`elements_Z.txt`	
`mp-2534/`	POSCAR (from MP), instructions.txt
`diagnosis_round1/`	`vasp_diagnose.py`: report.txt, fix/*.sh
`mp-149/`	Si
`mp-2534/`	GaAs, opened below: a project as in Example 2
`instructions.txt, POSCAR, POTCAR`	
`run_all.sh, makeplots.sh`	runs the steps; the plots
`01_relax/`	cell and ions (ISIF = 3)
`INCAR KPOINTS POSCAR POTCAR run.sh`	
`OUTCAR OSZICAR CONTCAR CHGCAR WAVECAR`	
`02_scf/`	charge density, then the ELF
`INCAR KPOINTS POSCAR POTCAR run.sh`	
`copy_from_relax.sh, run_elf.sh`	takes CONTCAR, CHGCAR; the ELF pass
`elf/`	short KPAR = 1 restart that writes ELFCAR
`OUTCAR CHGCAR ELFCAR vasprun.xml`	
`08_lobster/`	symmetry-off NSCF, then LOBSTER
`INCAR KPOINTS POTCAR run.sh lobsterin`	
`copy_from_scf.sh, lobster_basis.py`	takes CHGCAR; basis and NBANDS
`lobsterout *LIST.lobster *CAR.lobster`	spilling, bond integrals, curves
`plots/`	COHP, ELF and DOS plots
`round_1_<date>/, repair_history.tsv`	failed run saved; what was repaired
`mp-22862/`	NaCl

Figure 8: The folder of Example 4. Each material is a project folder of the agent, as in Example 2, named by its mp-ID. With `-agent slurm` on a cluster the folder is the same: `runall.sh` then submits every material, each with its own `submit_all.sh` and SLURM step scripts (logs `slurm-<jobid>.out` in the step folders). The cluster driver `ht_runs.py` of Example 5 puts the materials into a `materials/` folder instead (Figure 9).

The folder of a high-throughput run on a cluster (`ht_runs.py`, Example 5)

folder written at set-up written by the runs written by the repairs

`~/VASP-Flow/MY_HT_CALCS/`	made by `ht_runs.py -o MY_HT_CALCS`
`env.sh`	account, partition, walltimes, LOBSTER
`material_list.txt`	the mp-IDs
`materials_summary.csv`	formula, atoms, nodes per material
`potcar_manifest.json`	POTCAR variant of each element
`submit_all.sh, make_potcars.sh`	build POTCARs, submit the job chains
`status.sh, retune.sh`	progress; change the job sizes
`makeplots_all.sh, postprocess_all.sh`	plots and bond tables of all materials
`collect_results.sh`	one tarball of the results
`README_HPC.md, tools/`	instructions; plotting scripts
`diagnosis_round1/`	report.txt, diagnosis.tsv, fix/*.sh
`materials/`	one folder per material
`mp-149/`	Si
`mp-2534/`	GaAs, opened below
`POSCAR`	the primitive cell from MP
`instructions.txt`	the request given to the agent
`job.sbatch`	one job script, the step as argument
`makeplots.sh`	makes the plots in `plots/`
`01_relax/`	cell and ions (ISIF = 3)
`INCAR KPOINTS POSCAR POTCAR run.sh`	
`OUTCAR OSZICAR CONTCAR CHGCAR WAVECAR`	
`02_scf/`	self-consistent charge density
`INCAR KPOINTS POSCAR POTCAR run.sh`	
`copy_from_relax.sh`	takes CONTCAR and CHGCAR
`OUTCAR OSZICAR CHGCAR DOSCAR vasprun.xml`	
`03_bands/`	band structure
`INCAR KPOINTS POTCAR run.sh`	KPOINTS: the k-path
`copy_from_scf.sh`	takes CHGCAR
`OUTCAR EIGENVAL vasprun.xml`	
`08_lobster/`	symmetry-off NSCF, then LOBSTER
`INCAR KPOINTS POTCAR run.sh lobsterin`	
`copy_from_scf.sh`	takes CHGCAR
`lobster_basis.py, lobster_basis_table.json`	basis and NBANDS
`lobsterout`	charge spilling
`ICOHPLIST.lobster ICOBILIST.lobster`	bond integrals
`COHPCAR.lobster DOSCAR.lobster`	COHP and DOS curves
`basis_scan.tsv`	bases tried (spilling above 5 %)
`plots/`	band, COHP, ELF and DOS plots
`mp-2534-mp-2534_scf-<jobid>.out, .err`	SLURM log of each job
`round_1_<date>/`	failed run, saved before a repair
`repair_history.tsv`	what was repaired, round by round
`user_params.txt`	your settings (`set_params.sh`)
`mp-22862/`	NaCl
`...`	

Figure 9: The folder of Example 5 (here with Si, GaAs and NaCl). The POTCARs are built on the cluster by `submit_all.sh` from `potcar_manifest.json`; WAVECAR is deleted once LOBSTER has finished.

2. Optional: see an out-of-memory repair. Generate with the dense mesh of VASP-Flow before v2.5 and without the LOBSTER memory guard: add `--kspacing 0.10 --lobster-max-kpts 0 --lobster-max-kpw 0` to the command of step 1. The LOBSTER step of mp-972937 then gets a 24×24×24 mesh and stops at set-up with `insufficient virtual memory`.

3. Configure and submit. Edit the one settings file, then submit everything:

```
cd /scratch/$USER/VASP-Flow/HT_example5
vi env.sh                  # VASP module, LOBSTER_BIN (Linux build),
   ANALYSIS_PYTHON
./submit_all.sh            # builds the POTCARs, submits
   relax->scf->bands->lobster per material
./status.sh                # progress per step; squeue -u $USER for
   the jobs
```

4. Round 1, when the queue is empty. From the run folder:

```
bash ~/VASP-Flow/ht_tools/next_round.sh
```

It prints one line per failed material and asks before submitting anything, e.g.

```
HT_example5__mp-972937   relax:ok scf:ok bands:ok lobster:oom   oom
   -> lobster   cause 0.90  fix 0.85  round 1
HT_example5__mp-1080459  relax:relax_unconv scf:ok ...
   relax_unconv -> relax>scf>bands>lobster  cause 0.95  fix 0.80
   round 1
== changes per fix script:
-- HT_example5__mp-972937
   lobster: k-mesh 24x24x24 -> 12x12x12 (ISYM=0 memory)
Submit the 2 repair(s) now? [y/N]
```

Read `diagnosis_round1/report.txt` for the cause in words, the evidence and the certainty (Section 25.2.1), then answer `y`. The failed runs are kept in `materials/<id>/round_1_<date>/`. Which materials fail can differ on your cluster; the procedure is the same.

5. Round 2. When those jobs are done, run `next_round.sh` again. Repaired materials now show `ok`; repeat until it says `Nothing to submit.`

6. Your own setting for one material. Suppose you want the relaxation of mp-1080459 with a looser force criterion and twice the time:

```
bash ~/VASP-Flow/ht_tools/set_params.sh mp-1080459 relax
   "EDIFFG=-0.02; TIME=16:00:00"
bash ~/VASP-Flow/ht_tools/next_round.sh
```

The next round redoes that relaxation (and the steps after it) with your values (Section 25.4).

7. Results. When nothing is left to repair:

```
bash ~/VASP-Flow/ht_tools/audit_run.sh        # all ok? spilling?
   (read-only)
./postprocess_all.sh                          # bond tables ->
   screen_lobster_all.csv
./collect_results.sh                          # small
   screen_results.tar.gz for your computer
```

Open the unpacked tarball in the GUI with **Open results folder** (Section 4.7).

Part III.
Reference: inputs, methods and options

Every piece the examples used, and how to change it by hand. The automatic choices are meant to be left alone; each section says when it is worth changing them.

I want to …	Go to
know every field, button and panel of the GUI	Example 1 (Section 4)
know which files and scripts the agents create	Section 9
make or redraw the plots, change their energy window	Section 10
write or change `instructions.txt` (all keywords)	Section 11
choose the functional, spin, SOC or GGA+U	Section 12
test ENCUT and the k-mesh	Section 13
change the k-point mesh	Section 14
set any INCAR or KPOINTS tag myself	Section 15
get bands, DOS and ELF	Section 16
get LOBSTER bonds (COHP, COBI)	Section 17
get the dielectric constant and Born charges	Section 18
get Wannier functions or phonons	Sections 19, 20
set cores, nodes, KPAR/NCORE and walltimes	Section 21
see more `instructions.txt` examples (SOC, magnetism, phonons, pressure)	Section 22
set up my own high-throughput run and use its scripts	Sections 23, 24
know how a value was chosen (k-mesh, KPAR/NCORE, ALGO, …)	Appendix A

9. The command-line agents: commands and the files they create

9.1. On a workstation: `vasp-agent.py`

The agents read `POSCAR` and `instructions.txt` from the current folder (or the files given with `-i` and `-s`), create a project folder named by its `Project:` line, and generate the calculation steps inside it. Run the steps, and `makeplots.sh` for the plots, from that folder.

```
./vasp-agent.py
./vasp-agent.py -i my_inst.txt -s struct.vasp

cd ProjectName
./run_all.sh          # runs every step in sequence
./makeplots.sh        # plots after all steps complete
```

9.2. On a SLURM cluster: `vasp-agent-slurm.py`

```
./vasp-agent-slurm.py                  # uses profiles/slurm.json
./vasp-agent-slurm.py --profile slurm

cd ProjectName
./submit_all.sh        # sbatch with dependency chaining
```

```
squeue -u $USER          # monitor jobs
./makeplots.sh           # run locally after jobs finish
```

Two-phase workflow when convergence tests are requested:

```
./submit_convergence.sh      # STEP 1 -- submit convergence jobs
# review results to decide on what to set for ENCUT and KPOINTS,
    then:
./submit_calculations.sh     # STEP 2 -- prompts ENCUT/k-mesh, submits
```

9.3. What the agent creates

One project folder named by the `Project:` line, with one sub-folder per step, the input files, and the scripts that run the steps and make the plots: Figure 7.

Without the GUI on a cluster, first set the VASP paths and SLURM defaults in `profiles/slurm.json` (Section 3.5).

10. Making the plots: in the GUI or with `makeplots.sh`

The same plots can be made in two ways: in the browser (Results tab) or on the command line with the project's `makeplots.sh`. Both read the raw VASP and LOBSTER outputs and write PNG and PDF files into the project's `plots/` folder.

Plot	Made from	File in `plots/`
Relaxation: total energy per ionic step; with a volume relaxation (ISIF = 3, 6, 7, 8) also the pressure, with its target (PSTRESS or 0)	`01_relax/OUTCAR`, restarts included (`OUTCAR.timeout*`, `OUTCAR.pass*`; a dotted line marks each)	`*_relax_energy.*`, `*_relax_pressure.*`
Band structure	`03_bands/EIGENVAL` (sumo or `vasprun.xml` as fallback); spin up/down in two colours	`<project>_band.png/pdf`
Total and projected DOS	`04_dos/DOSCAR`; projections as asked in `instructions.txt` (`DOS (projected on Ga-p and As-p)`)	`<project>_dos.png/pdf`
ELF along bonds, ELF plane	`02_scf/ELFCAR` (only if the SCF wrote one)	`*_elf_bonds.*`, `*_elf_plane.*`
COHP, COBI, COOP	`08_lobster/*CAR.lobster`	`*_cohp.*`, `*_cobi.*`, `*_coop.*`
LOBSTER projected DOS	`08_lobster/DOSCAR.lobster`, with the `04_dos` total for comparison	`*_lobster_dos.*`
Bond table	ICOHP/ICOBI/ICOOP per bond, antibonding integrals	`lobster_summary.csv`
Convergence tests (energy, pressure, force vs. ENCUT and k-mesh; `makeplots_convergence.sh`)	the test points in `00_convergence/encut/`, `kpoints/`	`00_convergence/encut_*.png`, `kpoints_*.png` (not in `plots/`)

In the GUI the same plots are on the Results tab, with an energy-window bar and PDF buttons per group (Section 4.5); runs made elsewhere are opened with *Open results folder* (Section 4.7).

10.1. On the command line (`makeplots.sh`)

Every project made by `vasp-agent.py` or `vasp-agent-slurm.py` has an `makeplots.sh`. Run it in the project folder once the steps have finished — on a cluster, on the login node after the jobs:

```
cd GaAs_example
./makeplots.sh                # prints relax energy, E-fermi; writes
    the plots into plots/
./makeplots_convergence.sh    # the convergence-test plots (if there
    were tests)
```

It needs a `python3` with numpy and matplotlib (`pip install numpy matplotlib`); `sumo` is used when installed but is not required. It skips plots whose outputs are missing and can be rerun at any time (e.g. after a step was redone).

Another energy window or style. `makeplots.sh` is a short script that calls the plotters in `plots/`; edit its `-ymin -4 -ymax 4`, or call a plotter directly:

```
cd plots
python3 plot_band_eigenval.py ../03_bands --out GaAs_wide --ymin -8
    --ymax 6
python3 plot_dos.py ../04_dos --out GaAs --proj ../dos_proj.json
cd ../03_bands && sumo-bandplot --ymin -4 --ymax 4      # sumo's own
    band plot
```

High-throughput runs. In a run made by `ht_runs.py` every material's last job runs its `makeplots.sh` automatically (results in `materials/<id>/plots/`, log in `plots/makeplots.log`). If the cluster's `python3` lacks numpy or matplotlib, set `ANALYSIS_PYTHON` in `env.sh`; `./makeplots_all.sh` (or `./makeplots_all.sh mp-149`) redraws the plots for finished materials. With `ht-mp-scf.py` each material folder has its own `makeplots.sh`, run as above. Runs made before v2.6 have `analyze.sh` and `analysis/` instead; the GUI and the `ht_tools` scripts read both.

11. Describing a calculation in `instructions.txt`: every keyword

11.1. An example to start from

```
Project: My material name

Methods: PBEsol functional
         collinear magnetization
         GGA+U with U=4.0 on Fe-d orbitals

Tasks: structure relaxation
       SCF calculation
       band structure
       DOS (projected on Fe-d and O-p)

Convergence: Test k-points 6x6x6 to 12x12x12
             ENCUT 400-600 eV

MAGMOM: 4.0
NSW: 100
EDIFFG: -0.01
```

```
MPI: 128
NODES: 2
NTASKS_PER_NODE: 64
WALLTIME: 12:00:00
```

The parser is case-insensitive and searches the entire file, so section order is flexible.

11.2. All keywords

Keyword	Meaning
`Project:`	Project name; becomes the output folder.
`MAGMOM:`	Initial magnetic moments in μ_B: one number for all atoms (`4.0`, `-3.0`), per element (`Fe=4.0, O=0.6`) or a per-atom list (`2*4.0 2*-4.0`). Any `MAGMOM` line switches spin polarisation on; default 2.0 when only "collinear magnetization" is given (Section 12.3).
`ISIF:`	Relaxation type: 3=full, 2=ions only, 4=fixed volume.
`NSW:`	Max ionic steps. Default 100.
`EDIFFG:`	Force criterion in eV/Å. Default −0.01.
`ENCUT:`	Manual plane-wave cutoff in eV.
`KMESH:`	Explicit Gamma k-mesh overriding the automatic density, e.g. `8 8 8` or `12` (cubic). Applies to every automatic mesh (relax/SCF/DOS); for 2-D give the third value (`12 12 1`).
`KMESH_DENSITY:`	Automatic k-mesh density tier: `coarse` (1000 kpra, default for relaxation and convergence tests) or `fine` (6000 kpra, default for SCF/DOS/spectroscopic steps). kpra $= N_k \times N_{\text{atoms}}$ where $N_k = N_x N_y N_z$; subdivisions scale as $N_i \propto \lvert\mathbf{b}_i^*\rvert$. An integer is taken as the kpra target itself (also `KPRA:`, `KPOINTS_DENSITY:`). Overridden by explicit `KMESH` and by `KPOINTS` blocks.
`RELAX_KMESH_DENSITY:`	Density of the relaxation mesh (`coarse`, `fine` or an integer); default `coarse` whatever the SCF uses.
`PRESSURE:`	External pressure for constant-P relaxation, e.g. `10 GPa` or `50 kbar`. Forces `ISIF=3`, `IBRION=2`, `PSTRESS` (Section 15.4).
`INCAR: ... END_INCAR`	Block of literal INCAR tags merged into the generated INCAR(s); optionally per-step (Section 15.1).

Keyword (continued)	Meaning
`NKPTS:`	k-points per band segment. Default 40.
`MPI:`	Total MPI tasks (nodes × cores/node).
`KPAR:`	Global k-point parallelisation.
`NCORE:`	Global cores-per-orbital.
`SCF_KPAR:` *etc.*	Per-step KPAR: also BANDS_, DOS_, RELAX_, PHONONS_ (DFPT is forced to KPAR = NCORE = 1).
`SCF_NCORE:` *etc.*	Per-step NCORE; same prefixes.
`NODES:`	Compute nodes (SLURM).
`NTASKS_PER_NODE:`	MPI tasks per node (SLURM).
`PARTITION:`	SLURM partition.
`WALLTIME:`	One time limit for every step, e.g. `12:00:00` (default: per step, Section 21.3).
`RELAX_WALLTIME:` *etc.*	Per-step time limit (RELAX, SCF, BANDS, DOS, WANNIER, DFPT, PHONONS, LOBSTER).
`ACCOUNT:`	SLURM account / allocation.
`DFPT_EDIFF:`	EDIFF for DFPT. Default `1E-8`.
`PHONONS_DIM:`	Supercell, e.g. `2 2 2`.
`PHONONS_MESH:`	q-mesh, e.g. `20 20 20`.
`PHONONS_DISP:`	Displacement in Å. Default 0.01.
`PHONONS_BAND:`	High-symmetry path for phonon bands.
`PHONONS_NAC:`	`FALSE` to disable NAC.
`WANNIER_NUM_WANN:`	Number of Wannier functions.
`WANNIER_PROJ:`	Projections, e.g. `Ga:s;As:p`.
`WANNIER_EWIN:`	Energy window, e.g. `-5.0 to 15.0`.
`LOBSTER_MAX_KPTS:`	Largest $N_1N_2N_3$ of the symmetry-off LOBSTER mesh (default 10000, 0 = no limit).
`LOBSTER_MAX_KPW:`	Memory budget of the symmetry-off LOBSTER NSCF: largest k-points × plane waves; a denser mesh is reduced (default 6×10^6, 0 = no cap; Appendix A.4).
`LOBSTER_BASIS:`	`auto` (default; basis scan when the spilling is high), `min`, `standard`, `max`, `scan`, `off` (Appendix A.5).
`LOBSTER_SCAN_ABOVE:`	Charge spilling (%) above which `auto` scans the bases (default 5).
`LOBSTER_NBANDS_FACTOR:`	NBANDS of the LOBSTER NSCF = factor × the largest basis (default 1).
`REDUCE_CELL:`	`auto` (default: Niggli-reduce a skewed cell before the first step when LOBSTER runs), `on` (also without LOBSTER), `off` (keep the cell as given); alias `NIGGLI:` (Section 17).

12. Choosing functional, spin, SOC and GGA+U

12.1. Functionals

```
Methods: R2SCAN functional     # meta-GGA; also the default when no
   functional is named
Methods: PBE functional        # standard GGA
Methods: PBEsol functional     # GGA for solids
Methods: LDA functional
Methods: HSE06 functional      # hybrid -- more accurate but slower
Methods: RVV10 functional      # van der Waals
```

The functional is taken from a `Functional:` line if there is one, else from the `Methods:` line;

with neither, R2SCAN is used. ENCUT is 500 eV for every functional unless you give `ENCUT: ...` or run the ENCUT convergence test (worth doing for R2SCAN, which converges more slowly with ENCUT than the GGAs).

What R2SCAN adds to every INCAR: `METAGGA = R2SCAN`, `LASPH = .TRUE.` and `LMIXTAU = .TRUE.`; the last one also writes the kinetic-energy density to `CHGCAR`, which the fixed-density steps (bands, DOS, LOBSTER, Wannier, all `ICHARG = 11`) need with a meta-GGA. The Born-charge step uses the finite-field route (§18), because VASP's linear response does not support meta-GGAs. The automatic Hubbard U is 0.6 times the tabulated PBE value (§12.4). LDA is written as `GGA = CA` and built from the LDA POTCAR library (`VASP_POTCAR_DIR_LDA`, §3.4); the tag also makes sure VASP does not take the functional from a PBE POTCAR.

12.2. Spin-orbit coupling

```
Methods: SOC                                          # non-magnetic
    material
Methods: SOC with magnetization in z-direction # magnetic material,
    moments along z
```

Both use `vasp_ncl` and set `LSORBIT` and `LNONCOLLINEAR`. A plain `SOC` (also `SOC, non-magnetic`) writes `MAGMOM = 3N*0`: the run starts and stays without moments. This matters, because VASP's own default for a non-collinear run is 1 μ_B per atom along $(1, 1, 1)$, which lowers the symmetry and slows the run. The magnetization direction (`SAXIS`, and the moments along it) only means something for a magnetic material (Section 12.3); any mention of magnetization switches the moments on, so leave it out for a non-magnetic one.

12.3. Magnetic materials (collinear spin)

```
Methods: collinear magnetization
MAGMOM: 4.0
```

Sets `ISPIN = 2` and `MAGMOM = N * value` in every INCAR, where N is the atom count from the POSCAR. Three forms of the `MAGMOM:` line are understood (any of them switches spin polarisation on, with or without the words "collinear magnetization"):

```
MAGMOM: 4.0               # uniform; negative values keep their sign
MAGMOM: Fe=4.0, O=0.6     # per element (unlisted elements start at 0)
MAGMOM: 2*4.0 2*-4.0      # per atom in POSCAR order, n*value
    shorthand
```

A per-atom list must have exactly N entries; the agent stops with an explanatory message rather than silently falling back to a default.

> **Tip.** Typical starting moments: Fe, Co ≈ 4–5 μ_B; Ni ≈ 2 μ_B; Cr, Mn ≈ 3–4 μ_B; rare earths ≈ 7 μ_B. VASP converges the self-consistent value automatically.

Antiferromagnets — give the alternating signs directly, e.g. `MAGMOM: 4.0 -4.0 4.0 -4.0`.

With SOC (`Methods: SOC with magnetization in x-direction`) the moments are $3N$ components in the non-collinear `MAGMOM`; the same `MAGMOM:` values are rotated onto the requested direction (without this VASP would start every moment along its default $(1, 1, 1)$). A list of exactly $3N$ numbers is used as the raw x, y, z components.

Spin-resolved plots produced automatically after DOS/bands run:

- *Cumulative DOS* — spin-up above the x-axis; spin-down mirrored below zero; elements colour-coded; solid = ↑, dashed = ↓.
- *Band structure* — spin-up in blue, spin-down in red dashed, same k-axis, $E_F = 0$.

- *Projected DOS* — sumo natively negates the spin-down channel in the orbital-resolved PDF.

Check the converged moment:

```
grep "magnetization␣(x)" OUTCAR | tail -5
```

12.4. GGA+U

```
Methods: GGA+U with U=4.0 on Fe-d orbitals
         GGA+U with U=3.0 on Ni-d orbitals
         GGA+U with U=6.0 on Ce-f orbitals
```

Dudarev scheme (LDAUTYPE=2); $U_{\text{eff}} = U - J$ with $J = 0$ set automatically. In the Dudarev formalism the parameter labelled U is already U_{eff}, so no separate J entry is needed. Combine freely with `collinear magnetization`.

Default values come from `hubbard_u_defaults.csv` (repo root: element, orbital, $U_{\text{eff}} = U - J$, ...; 3d, 4d, 5d, 4f, 5f elements) — the only source of default values (there are no built-in ones). Dudarev's scheme uses U_{eff} only, so the INCAR gets `LDAUU` $= U_{\text{eff}}$ and `LDAUJ` $= 0$ (a J column in the file is ignored); rows with $U_{\text{eff}} = 0$ (Zn, Cd) get no correction. These values were fitted for oxides, chalcogenides and halides, so their use is set by one optional flag in `instructions.txt`:

Flag	Effect
(none)	**default**: GGA+U on the tabulated d/f elements only if the POSCAR contains a chalcogen or halogen (O, S, Se, Te, F, Cl, Br, I); otherwise $U = 0$. With R2SCAN the value is $0.6\,U_{\text{eff}}$ (the table is for PBE); with HSE06 no automatic U.
`GGA_U: ON`	tabulated U_{eff} on every tabulated element, whatever the compound.
`GGA_U: OFF`	no U (also `no GGA+U`).
`GGA_U: AUTO`	same as no flag.
`GGA+U with U=... on El-orb`	explicit values; always win and are used as given (not scaled for R2SCAN).

The key may also be written `GGA-U`, `GGA+U` or `GGAU`, and key and value are case-insensitive (`gga+u: on`); likewise the command-line option of the high-throughput drivers accepts `--gga_u`, `--gga-u`, `--gga+u` (or upper case) with `auto`, `on`, `off` in any case.

The same U block is written to every step (relax, SCF, bands, DOS, LOBSTER, ...), so all steps share one Hamiltonian. R2SCAN already removes part of the self-interaction error that U corrects, so its automatic U is 0.6× the tabulated PBE value (`R2SCAN_U_FACTOR` in `modules/vasp_input_generator.py`; e.g. Fe 4.1 → 2.46 eV, written in the INCAR comment); none is automatic under HSE06. All three routes honour the flag: the GUI (Setup, *Default Hubbard U*: auto / on / off), `vasp-agent.py` and `vasp-agent-slurm.py`; the high-throughput drivers write it with `--gga_u auto|on|off`.

13. Convergence tests for ENCUT and k-points: syntax and outputs

```
# Range (auto-steps)
Convergence: Test k-points from 6x6x1 to 18x18x1    # 2D material
             Test k-points from 6x6x6 to 12x12x12   # 3D material
             ENCUT 400-600 eV

# Explicit mesh list
```

```
Convergence: Test k-points 6x6x3, 12x12x6, 18x18x9

# ENCUT: all of these are equivalent (step defaults to 50 eV)
Test ENCUT 350-650      Test ENCUT 350 650      Test ENCUT 350 to 650
Test ENCUT 350,650      Test ENCUT 350:50:650   # start:step:stop
```

The `start:step:stop` form is the only one that sets the step; the others sweep in 50 eV increments. A convergence line that cannot be parsed now emits a `[warn]` on stderr rather than silently generating no tests at all.

Convergence INCARs are trimmed to what the test actually reads — the total energy in `OUTCAR`. `LWAVE`, `LCHARG`, `LORBIT` and `LELF` are dropped, which avoids writing a full WAVECAR per point and, because `LELF` pins KPAR to 1, restores k-point parallelism for the sweep.

Results in `00_convergence/kpoints/` and `00_convergence/encut/`.

```
./run_convergence.sh          # workstation
./submit_convergence.sh       # SLURM
./makeplots_convergence.sh      # plots: energy / pressure / forces
```

After the runs the smallest ENCUT and k-mesh that meet the force and stress criteria are written to `00_convergence/converged_params.txt` and offered as defaults for phase 2 (Appendix A.9).

14. Choosing the k-point mesh: densities, fixed meshes, DOS and LOBSTER

All Gamma-centred meshes are generated automatically from the POSCAR geometry. **Leave them automatic unless** a convergence test (Section 13) shows the energy or forces still changing, or you must match the mesh of another calculation. Two density tiers are available via the `KMESH_DENSITY` key (Section 11.2):

Tier	kpra	Typical use	Notes
`coarse`	1000	Relaxation, convergence tests	Agent k-point sweeps start here
`fine`	6000	SCF, DOS, spectroscopic steps	Default for all production steps

Each axis gets a number of points proportional to its reciprocal-lattice length, so the spacing is the same in all directions; the formula is in Appendix A.1. The first line of each generated KPOINTS reports the mesh and the density reached. Any integer may be given instead of `coarse`/`fine` (`KMESH_DENSITY: 3000`); the relaxation has its own density (`RELAX_KMESH_DENSITY`, default coarse).

DOS mesh: always $2N_x \times 2N_y \times 2N_z$ (the SCF ratios are preserved; a 2-D slab keeps $N_z = 1$), for accurate spectral features.

LOBSTER mesh: the SCF mesh. The LOBSTER NSCF runs with `ISYM`=0, so it pays for every k-point of the mesh, not the irreducible wedge. It is therefore **kept within limits automatically**: if the mesh has more than `LOBSTER_MAX_KPTS` points ($N_1N_2N_3$, default 10000, about 20×20×20) or its k-points times plane waves exceed `LOBSTER_MAX_KPW` (6×10^6, the memory limit), it is reduced, keeping its ratios, until both hold; the KPOINTS comment says so (Appendix A.4). The DFPT mesh is forced to match the SCF mesh exactly.

An explicit `KMESH: N1 N2 N3` entry overrides the automatic density for all steps, and a `KPOINTS` block (Section 15.2) overrides everything for the step it names.

15. Setting INCAR and KPOINTS tags yourself

Leave the generated INCARs alone unless you need a tag VASP-Flow does not set, or a different value than it chose (smearing, EDIFF, your own U); the overrides are written under a comment in the INCAR, so they stay visible. **Any VASP keyword can be specified explicitly** — nothing is hard-wired. Three routes, in increasing precedence: (1) an

`INCAR: ... END_INCAR` block in the instructions file (global or per-step); (2) the agents' `-incar` command-line option; (3) direct editing — in the GUI every generated INCAR is editable per step (Workflow tab → *Edit files*, with an automatic `.bak` backup), and of course the files on disk can be edited before running. This section covers the first two plus **constant-pressure relaxation**.

15.1. Raw INCAR passthrough

Place literal INCAR lines — in ordinary VASP syntax — inside an `INCAR: ... END_INCAR` block in your `instructions.txt`. They are merged into the generated INCAR(s):

```
INCAR:                    # global block -- applies to every step
   LREAL  = Auto
   ALGO   = Fast
   NELMIN = 6
END_INCAR
```

A tag that the agent already writes is **overwritten in place** (so `ALGO = Fast` above replaces the default `ALGO = Normal`); any tag the agent does not normally emit is appended under a clearly labelled `# User INCAR overrides` section, so nothing is silently lost.

To target a single step, name it on the header line. Recognised steps are `relax`, `scf`, `bands`, `dos`, `wannier`, `dfpt`, `phonons`, `lobster`:

```
INCAR scf:                # applies only to 02_scf
   EDIFF = 1E-8
   SIGMA = 0.10
END_INCAR
```

Note. Precedence: a *per-step* block beats the *global* block, and both beat the agent's generated defaults. This is the recommended way to set tags the parser has no dedicated keyword for (e.g. `LMAXMIX`, `ADDGRID`, `LVTOT`, `IVDW`, `AMIX`). Full-line `#` comments in the instructions file are ignored by the parser, so a task keyword inside a comment does not enable that task.

15.2. Explicit KPOINTS blocks

The same block syntax writes a complete KPOINTS file, *unchanged*, for a step:

```
KPOINTS scf:              # or KPOINTS_SCF:
   my SCF mesh            # first line = KPOINTS comment
   0
   Gamma
   9 9 5
   0 0 0
END_KPOINTS
```

Steps: `relax`, `scf`, `bands`, `dos`, `wannier`, `dfpt`, `phonons`, `lobster`; an unnamed `KPOINTS:` block applies to every step except the band path. Explicit blocks (INCAR and KPOINTS alike) **override every other setting**: keywords, `KMESH`, the density tier and the automatic `KPAR`/`NCORE`. Steps derived from the SCF mesh — DOS and LOBSTER (2×), DFPT and Wannier90 (1×) — follow an explicit SCF block. A marker file `.explicit_kpoints` is left beside such a KPOINTS so that the convergence phase-2 k-mesh patching never overwrites it. `KPAR` and `NCORE` written in an INCAR block also steer the companion value (e.g. `KPAR = 2` on 16 ranks gives `NCORE = 2`). In the GUI both block kinds are entered under *Setup* → *Advanced — explicit INCAR / KPOINTS*.

15.3. Command-line INCAR overrides (`-incar`)

Both `vasp-agent.py` and `vasp-agent-slurm.py` accept a repeatable `-incar` option that injects explicit VASP tags at invocation time, without touching the instructions file:

```
./vasp-agent.py -i instructions.txt -s POSCAR \
    --incar "ALGO=Fast;␣NELMIN=6" \      # every step
    --incar "scf:NBANDS=64"              # 02_scf only
```

Multiple tags in one option are separated by `;` (not commas — values such as `MAGMOM` may contain commas). An optional step prefix (`relax/scf/bands/dos/wannier/dfpt/phonons/lobster`) targets a single step. Command-line tags are applied *after* the instructions-file blocks, so they win on conflicts.

Note. Exceptions: physics-critical guards are re-imposed regardless of overrides — `ISYM=2` and `KPAR=NCORE=1` in `06_dfpt`, and the symmetry-off `ISYM` in `08_lobster`.

15.4. Constant-pressure relaxation

A single `PRESSURE` line turns the relaxation into a constant-pressure optimisation: the agent forces `ISIF = 3` (cell shape + volume + ions) and `IBRION = 2`, and imposes the target pressure through `PSTRESS`.

```
PRESSURE: 10 GPa          # also accepted: "5 kbar", "constant
    pressure 8"
```

VASP's `PSTRESS` tag is in **kBar** (1 GPa = 10 kBar). The agent converts automatically; if no unit is given, **GPa is assumed**. The resulting `01_relax/INCAR` reads, e.g.:

```
# Constant-pressure relaxation @ 10 GPa  (PSTRESS = 100 kBar)
IBRION = 2
ISIF   = 3
NSW    = 100
EDIFFG = -0.01
PSTRESS = 100   ! external pressure in kBar (10 GPa)
```

Tip. `PSTRESS` adds the PV term so the relaxed cell sits at the requested external pressure. Use a tight `EDIFFG` and a well-converged `ENCUT`; the Pulay stress error grows at high pressure, so verify with a higher `ENCUT` before trusting the equilibrium volume.

How the SCF starts from the relaxation (which files are copied, `ISTART`/`ICHARG`) is described in Appendix A.6; which eigensolver each step uses in Appendix A.3.

16. Band structures, DOS projections and ELF profiles

16.1. Band-structure k-path

16.1.1. Auto-detection (recommended)

Omit the path in `instructions.txt`; the GUI fills its *K-path* field with the same spglib path as soon as a POSCAR is pasted or loaded (and ticks *Hexagonal BZ* for a hexagonal cell), so there is nothing to choose:

```
Tasks: band structure          # no "along ..." -> auto-detect from
    POSCAR
```

The path is the standard Setyawan–Curtarolo path of the actual space group (found with spglib), or a geometric guess when spglib is missing; details and the table of default paths are in Appendix A.7.

Note. For **conventional** FCC or BCC cells (right angles, 4 or 2 atoms), auto-detection returns the cubic (cP) path. Use the **primitive** cell for an unfolded FCC/BCC band structure — the primitive FCC cell has angles ≈ 60° and is recognised automatically.

The detection route appears in the KPOINTS comment:

```
Line-mode KPOINTS  [spglib ... -- auto-detected]   # spglib route
Line-mode KPOINTS  [FCC -- auto-detected]          # geometric
   fallback
```

16.1.2. Manual override

```
Tasks: band structure along G-X-W-K-G-L       # '-' joins points
Tasks: band structure along G-X-W-K-G-L|U-X   # '|' starts a new
   segment
```

The labels get their coordinates from spglib's points for this cell (KPOINTS comment `[spglib FCC points, path as given]`) when spglib knows all of them; otherwise from the generic table below, which is right only for the simple cells it names:

Label	Coords	Use	
G	(0, 0, 0)	all	Γ
M	(0.5, 0.5, 0)	cubic, tet	(0.5, 0, 0) in hexagonal
K	(1/3, 1/3, 0)	hexagonal	
R	(0.5, 0.5, 0.5)	cubic	
Z	(0, 0, 0.5)	general	
A	(0, 0, 0.5)	hex, tet	
L	(0.5, 0, 0.5)	hexagonal	
H	(1/3, 1/3, 0.5)	hexagonal	
S	(0.5, 0.5, 0)	orthorhombic	
W	(0.5, 0.25, 0.75)	**FCC primitive**	
P	(0.25, 0.25, 0.25)	**BCC primitive**	
N	(0, 0, 0.5)	**BCC primitive**	

16.2. Projected DOS

All of the following forms are accepted:

```
DOS (projected on Mo-d and S-p)
DOS with projections on Ni:d and Ni:s
DOS with projection on Fe-d and O-p
```

Orbital channels: `s`, `p`, `d`, `f`. Separator is either `-` or `:`. Sumo generates an orbital-resolved PDF in `plots/`.

16.3. ELF bond profiles

If an SCF calculation writes an `ELFCAR` (electron localization function, requested with `LELF = .TRUE.` — the high-throughput driver sets this automatically), the utility `elf_bonds.py` plots ELF along each non-equivalent nearest-neighbour bond, sampled from one atom centre to the other:

```
./elf_bonds.py [ELFCAR] [--out PREFIX] [--nn-scale 1.2] [--npoints
   200]
```

It parses the `ELFCAR` grid directly (only numpy + matplotlib), groups bonds by element pair and length (rounded to 0.01 Å) to pick the non-equivalent ones, and interpolates ELF along each

bond with periodic boundary conditions. Output is `FORMULA_elf_bonds.png/pdf` (the prefix defaults to the reduced formula, e.g. `MoSe2_elf_bonds.pdf`; override with `-out`) showing ELF vs. distance along the bond, with the ELF = 0.5 homogeneous-electron-gas reference marked. `makeplots.sh` runs it automatically when `02_scf/ELFCAR` is present, and the GUI Results tab shows the same plot whenever the SCF wrote an `ELFCAR`.

Tip. Reading the profile: ELF is high (near 1) where electrons are paired and localized. A covalent bond therefore shows an ELF *maximum* in the bonding region *between* the atoms (e.g. Si–Si ≈ 0.95 at mid-bond), not at the nuclei. Because VASP builds the `ELFCAR` from the pseudo (valence) density, the value *at* an atom centre is a PAW artifact — often low, even ≈ 0 for heavy atoms — so read the bonding region, not the endpoints.

Note. VASP does *not* compute ELF for non-collinear / spin-orbit runs (`LSORBIT`/`LNONCOLLINEAR`); it prints `"WARNING: ELF not implemented for non collinear case"` and writes an all-zero `ELFCAR`. `elf_bonds.py` detects this and reports it rather than producing a blank plot — re-run the SCF *without* SOC to get ELF.

ELF without slowing down the SCF. `LELF` needs KPAR = 1. With `ELF: separate` (or `ELF: two-step`) in `instructions.txt` the SCF runs at full KPAR and a short KPAR = 1 restart in `02_scf/elf/` writes the ELFCAR (Appendix A.8); `ht-mp-scf.py` does this by default.

17. LOBSTER bonding analysis (COHP, COBI, COOP)

Adding `LOBSTER COHP/COBI analysis` (keywords: `lobster`, `cohp`, `cobi`, `coop`, `bonding`) to the task list creates `08_lobster`: a symmetry-off NSCF that reads `02_scf/CHGCAR`, followed by a run of the `lobster` binary.

- **ISYM=0 is required** — a symmetrised k-mesh gives unusable COHP/ICOHP values (wrong sign, errors above 100%). The generated INCAR sets it automatically. With SOC enabled, **ISYM=−1** is used instead: spin-orbit coupling breaks time-reversal symmetry, which ISYM=0 would still assume.
- The k-mesh is the SCF mesh, reduced automatically above 10000 points or when the symmetry-off run would not fit in memory (`LOBSTER_MAX_KPTS`, `LOBSTER_MAX_KPW`, Appendix A.4).
- `NBANDS` is auto-set to at least the LOBSTER basis-function count; `LMAXMIX` is chosen from the elements (6 for *f*, 4 for *d*, 2 for *sp*).
- The **charge spilling** (in `lobsterout`) measures how well LOBSTER's atomic basis represents the bands; when it is above 5 % the step reruns LOBSTER with the other tabulated bases and keeps the best (`LOBSTER_BASIS`; Appendix A.5).
- **Skewed cells are reduced first.** In a long, thin, skewed cell LOBSTER's projection breaks down: the Materials Project primitive cell of 15R-SiC (mp-11713, three 12.7 Å vectors 13.9° apart) gave 18–58 % charge spilling with every basis, and its Niggli-reduced cell (3.08, 3.08, 12.71 Å; the same crystal) 1.5 %. When LOBSTER is requested, the agents therefore replace such a cell by its Niggli-reduced cell before the relaxation, so every step runs on it; the original is kept as `POSCAR.input`, and the agent prints both cells. A cell that is already reduced is left exactly as given (axis order included). `REDUCE_CELL: off` (GUI: untick *Niggli-reduce a skewed cell*) keeps a skewed cell, with a warning; the cell

is also kept, with the same warning, when `KMESH:` or a `KPOINTS` block is given for it, or for a 2-D slab. `ht_runs.py` reduces at staging (`--no-reduce-cell` to keep the cell as downloaded).

- The LOBSTER binary is set with `LOBSTER_X` in `default_paths` (default `lobster-5.1.0-OSX`); it can be overridden at runtime via `$LOBSTER_BIN`.

```
Project: NaCl
Methods: PBE functional
Tasks: SCF, LOBSTER COHP/COBI analysis
MPI: 16
```

Run with `./vasp-agent.py` (builds and runs `02_scf` + `08_lobster`); the GUI shows COHP/COBI/COOP plots on the Results tab. To aggregate averaged COHP/COBI over many projects into `lobster_acohp_acobi.csv`, run `python3 lobster_postprocess.py`.

Why the LOBSTER projected DOS and the VASP DOS differ is explained in Appendix A.11.

18. Dielectric constant and Born charges (DFPT)

The `DFPT` task creates `06_dfpt`, which computes Born effective charges Z^*, the macroscopic dielectric tensor, and the piezoelectric stress tensor from the converged SCF charge density.

Two routes are generated automatically depending on the method:

Method	Route
PBE, PBEsol, LDA, AM05 (no SOC)	Linear response: `IBRION=8`, `LEPSILON=.TRUE.`
SOC, HSE06, or R2SCAN	Finite field (PEAD): `IBRION=6`, `LCALCEPS=.TRUE.` — VASP's DFPT is not implemented for noncollinear magnetism, hybrids, or meta-GGAs. Requires an insulating system.

KPAR, NCORE and ISYM are fixed for this step (Appendix A.10).

`run.sh` executes `extract_born.py` after VASP, which writes:

- `BORN` — phonopy NAC format: electronic ε_∞ + Z^* per atom.
- `born_charges.txt` — human-readable summary shown in the GUI Results tab: electronic (ion-clamped) ε_∞, ionic contribution, total static $\varepsilon_0 = \varepsilon_\infty + \varepsilon_{\text{ion}}$; Z^* tensors labelled by element; an acoustic-sum-rule check ($\sum_{\text{ions}} Z^*$ should vanish — a large residual indicates an under-converged k-mesh or EDIFF); and the piezoelectric stress tensors e (C/m^2, clamped-ion, ionic, and relaxed-ion total; meaningful only for non-centrosymmetric crystals).

`DFPT_EDIFF` in `instructions.txt` tightens the electronic convergence (default `1E-8`).

18.1. Example

```
Project: GaAs
Methods: PBE functional
Tasks: relaxation, SCF, DFPT Born charges and dielectric
DFPT_EDIFF: 1E-8
MPI: 16
```

In the GUI, tick **Born Charges & Dielectric Constant (DFPT)** on the Setup tab (the DFPT EDIFF field is optional); after running `06_dfpt` from the Workflow tab, the full summary appears in the **Born charges** panel of the Results tab. With SOC, HSE06 or R2SCAN selected

under Methods, the finite-field INCAR is generated automatically — no extra input is needed. For LO-TO splitting in phonon spectra, combine with the phonon task as in the GaAs worked example (Section 22.3).

19. Wannier functions (Wannier90)

```
Tasks: Wannier90 wannierization

WANNIER_NUM_WANN: 8
WANNIER_PROJ: Ga:s;As:p
WANNIER_EWIN: -5.0 to 15.0
```

The agent generates `05_wannier/wannier90.win` with the NSCF k-mesh and projection block. After the NSCF run, execute `wannier90.x` from the `05_wannier/` directory.

20. Phonons (phonopy)

```
Tasks: phonons lattice dynamics

PHONONS_DIM: 2 2 2
PHONONS_MESH: 20 20 20
PHONONS_DISP: 0.01
PHONONS_BAND: G M K G A
PHONONS_NAC: FALSE      # disable non-analytic correction
```

Note. For the non-analytic correction (important for polar materials), include `DFPT` in your task list so Born effective charges are computed.

21. Cores, nodes, KPAR/NCORE and walltimes

21.1. KPAR and NCORE

KPAR and NCORE are set automatically per step from the number of MPI ranks and the step's k-points (rule in Appendix A.2). **Leave them automatic unless** a step runs out of memory (lower KPAR) or you have timed a faster pair on your machine. To set them yourself:

```
MPI: 128        KPAR: 4        NCORE: 8

# Per-step (take priority over globals)
SCF_KPAR: 4     BANDS_KPAR: 1     DOS_KPAR: 8
SCF_NCORE: 8    BANDS_NCORE: 16   DOS_NCORE: 8
RELAX_KPAR: 2   PHONONS_KPAR: 1   # DFPT is always KPAR=1, NCORE=1
```

21.2. Per-project resources (in `instructions.txt`)

The cluster defaults (partition, nodes, tasks per node, account) are in `profiles/slurm.json` (Section 3.5); these lines override them for one project:

```
NODES: 4     NTASKS_PER_NODE: 32     PARTITION: gpu     ACCOUNT:
   myallocation
RELAX_WALLTIME: 12:00:00      # one step
WALLTIME: 48:00:00            # every step (instead of the per-step
   defaults)
```

21.3. Walltime per step

Each step is its own SLURM job with its own time limit, so short steps get short limits (and better queue priority). Defaults:

Step	Walltime
`01_relax`	08:00:00
`02_scf`	04:00:00
`03_bands`, `04_dos`	02:00:00
`08_lobster`	04:00:00 (symmetry-off NSCF over the full mesh, plus the LOBSTER projection)
others	the profile time

Priority: `<STEP>_WALLTIME` > `WALLTIME` > these defaults > `profiles/slurm.json`. In `ht_runs.py` directories the same defaults sit in `env.sh` (`TIME_RELAX ...`; `retune.sh --time-relax ...`); that is the only place to change them — the per-material `instructions.txt` carry no walltime.

A step that reaches its limit is stopped cleanly 15 minutes before and continued automatically in a requeued job (Appendix B).

22. More `instructions.txt` examples: SOC, magnetism, phonons, pressure

22.1. MoS_2 monolayer with SOC

```
Project: MoS2 monolayer

Methods: PBEsol functional
         SOC                  # non-magnetic: MAGMOM = 3N*0

Tasks: structure relaxation
       SCF calculation
       band structure
       DOS (projected on Mo-d and S-p)

Convergence: Test k-points 8x8 to 20x20
             ENCUT 400-600 eV

MPI: 64
```

The generator detects hexagonal (hP) and uses G-M-K-G-A-L-H-A.

22.2. Nickel: magnetism + GGA+U

```
Project: Nickel GGA+U

Methods: PBEsol functional
         collinear magnetization
         GGA+U with U=4.0 on Ni-d orbitals

Tasks: structure relaxation
       SCF calculation
       band structure
       DOS (projected on Ni-d and Ni-s)

MAGMOM: 2.0
MPI: 128
```

```
NODES: 2
NTASKS_PER_NODE: 64
WALLTIME: 08:00:00
```

With a primitive FCC POSCAR (angles $\approx 60°$) the generator detects cF and uses G-X-W-K-G-L-U-W. Band and DOS plots show both spin channels.

22.3. GaAs: phonons with NAC

```
Project: GaAs phonons

Methods: PBEsol functional

Tasks: SCF calculation
       DFPT Born charges and dielectric
       phonons lattice dynamics

PHONONS_DIM: 4 4 4
PHONONS_MESH: 20 20 20
PHONONS_BAND: G X W L G K

MPI: 64     WALLTIME: 24:00:00
```

22.4. High-pressure relaxation with custom INCAR tags

A constant-pressure cell relaxation at 10 GPa, combined with raw INCAR passthrough to tighten convergence and add tags the parser has no dedicated keyword for:

```
Project: Silicon 10GPa

Methods: PBEsol functional

Tasks: structure relaxation
       SCF calculation
       band structure

PRESSURE: 10 GPa          # constant-P relax -> ISIF=3, IBRION=2,
   PSTRESS=100

# Tags applied to every step
INCAR:
   LREAL  = .FALSE.
   ADDGRID = .TRUE.
   LMAXMIX = 4
END_INCAR

# Tighter electronic convergence for the relaxation only
INCAR relax:
   EDIFF  = 1E-8
   EDIFFG = -0.005
END_INCAR

ENCUT: 600
MPI: 64
```

The `01_relax/INCAR` is generated with `ISIF=3`, `PSTRESS=100`, the global tags appended, and `EDIFF`/`EDIFFG` overridden to the per-step values; `02_scf` and `03_bands` receive only the global

block.

22.5. Bi_2Se_3: topological insulator

```
Project: Bi2Se3 topological

Methods: PBEsol functional
         SOC                  # non-magnetic: MAGMOM = 3N*0

Tasks: structure relaxation
       SCF calculation
       band structure
       DOS (projected on Bi-p and Se-p)
       Wannier90 wannierization

Convergence: Test k-points from 8x8x4 to 16x16x8
             ENCUT 400-600 eV

WANNIER_NUM_WANN: 18
WANNIER_PROJ: Bi:p;Se:p
WANNIER_EWIN: -2.0 to 2.0
MPI: 128
```

23. High-throughput runs: the two drivers and their setup on a cluster

Two drivers turn a list of Materials Project IDs (one `mp-id` per line; blank lines and `#` comments are ignored) into VASP runs. Both download the **primitive** cell from MP and use the same agents and input generator as the rest of VASP-Flow.

	`ht-mp-scf.py`	`ht_runs.py`
Steps per material	`01_relax` (ISIF = 3) → `02_scf` + ELF → `08_lobster` (optional)	`01_relax` (ISIF = 3) → `02_scf` → `03_bands` → `08_lobster` (no ELF, no DFPT)
Runs on	workstation or SLURM	SLURM only
Jobs	one job per step, submitted by `runall.sh`	one job per step (`job.sbatch <step>`), chained by `submit_all.sh`
Walltime	per step (Section 21.3)	per step, `TIME_*` in `env.sh`
Resources	one setting for all materials	ENCUT (1.3× max ENMAX) and job size per material
Extras	element/Z table	throttled, resumable `submit_all.sh`; `status.sh`, `retune.sh`, `makeplots_all.sh`, `collect_results.sh`; POTCAR manifest
Use for	structures to relax and characterise (ELF, bonding)	any list of materials (metals, semimetals, insulators) for bands and bonding; repaired round by round with `next_round.sh`

23.1. Step by step on the cluster (Rivanna)

1. Every new login: update the code and set the environment. The drivers download from MP, so run them on the login node (internet access).

```
cd ~/VASP-Flow && git pull
source ~/vaspenv/bin/activate     # made once at installation
export MP_API_KEY=your_key        # materialsproject.org/api
export VASP_POTCAR_DIR=/path/to/PAW_PBE
```

2. Put the list in a file, e.g. `~/VASP-Flow/lists/mylist.txt`.

3a. Relax, SCF, bands and LOBSTER (`ht_runs.py`):

```
cd ~/VASP-Flow
python3 ./ht_runs.py --list lists/mylist.txt \
    --out HT_mylist --api-key "$MP_API_KEY" --single-node
cd HT_mylist
vi env.sh                         # VASP module, LOBSTER_BIN,
   ANALYSIS_PYTHON
./retune.sh --time-lobster 06:00:00    # optional: change a step
   walltime
./submit_all.sh                   # POTCARs + submit; re-run = resume
./status.sh                       # progress per step
# when the queue is empty: repair the failed runs, round by round
   (Part IV)
bash ~/VASP-Flow/ht_tools/next_round.sh
# when nothing is left to repair:
./postprocess_all.sh              # -> screen_lobster_all.csv
./collect_results.sh              # -> screen_results.tar.gz
```

3b. Relax + SCF + ELF (`ht-mp-scf.py`). It writes the material folders into the directory it is run from, so use a work directory:

```
mkdir -p /scratch/$USER/ht_mylist && cd /scratch/$USER/ht_mylist
python3 ~/VASP-Flow/ht-mp-scf.py -l ~/VASP-Flow/lists/mylist.txt \
    --agent slurm --no-chain --nodes 1 --ntasks-per-node 40 \
    --account elmgroup --kpra 8000 --encut 520
./runall.sh      # build inputs, submit relax -> SCF+ELF -> LOBSTER
squeue -u $USER
```

Check `~/VASP-Flow/profiles/slurm.json` first: its `modules` list must load a working VASP on the cluster (on Rivanna `vasp/5.4.4` alone; extra compiler/MPI modules cause version swaps). Choose `--encut` as about 1.3× the largest ENMAX in the set (the ISIF = 3 relaxation needs it against Pulay stress). `--no-chain` lets the materials run in parallel; the default (chained) runs everything one after another, and one failed step then blocks all later jobs.

POSCARs already downloaded? Neither driver downloads a structure it already has. `ht_runs.py` reuses `<out>/_ht_inputs/<id>/POSCAR`, `ht-mp-scf.py` the `_ht_inputs/` of the current directory. To take them from another folder — e.g. an older highthroughput directory — add `--poscar-dir <folder>` (repeatable; it looks for `<id>/POSCAR`, `materials/<id>/POSCAR`, `_ht_inputs/<id>/POSCAR` or `<id>.vasp`). No MP key or internet is needed then; only missing structures are downloaded.

```
python3 ./ht_runs.py --list lists/mylist.txt --out HT_new \
    --poscar-dir HT_mylist --single-node
```

4. Test first. Run two or three materials before a long list, then read the timings and look at the plots (the per-step walltimes of Section 21.3 are defaults; steps that hit them continue automatically):

```
grep "Elapsed time" materials/*/0*/OUTCAR      # ht_runs
grep "Elapsed time" */0*/OUTCAR                # ht-mp-scf
```

5. Look at the results in the GUI (Section 4.7): unpack `screen_results.tar.gz` on your computer (or run the GUI on the cluster with `ssh -L 5001:localhost:5001`) and use **Open results folder**; missing plots are drawn there.

A complete small run with repair rounds is Example 5 (Section 8). All options of the two drivers are listed in Appendix C. When the jobs have finished, check and repair the runs as described in Part IV.

24. Scripts inside a high-throughput directory: submit, status, retune, collect

A directory made by `ht_runs.py` contains these scripts; run them from that directory.

Script (in the directory)	Purpose
`env.sh`	The one file to edit on the cluster: modules, `VASP_STD`, `LOBSTER_BIN`, `VASP_POTCAR_DIR`, `ANALYSIS_PYTHON`, step walltimes `TIME_*`, `VF_MAX_RESTARTS`, disk policy.
`submit_all.sh [N]`	Build missing POTCARs, submit each material's unfinished steps as a chain (at most `MAX_QUEUED`=200 jobs queued); skips finished/queued materials, so it is also the resume command.
`status.sh`	Per-step completion counts; writes `incomplete.txt`.
`retune.sh`	Change the step walltimes in `env.sh` (`--time-relax`, `--time-scf`, `--time-bands`, `--time-lobster`, or `--time` for all) or partition/account in every job script.
`make_potcars.sh`	Build and verify the POTCARs (run automatically).
`makeplots_all.sh [id]`	Redo the plots (each job already runs `makeplots.sh`).
`postprocess_all.sh`	Per-material and combined LOBSTER bond tables.
`collect_results.sh`	Small `screen_results.tar.gz` with the per-step layout (trimmed OUTCARs; `FULL_OUTCAR=1` for full).

Part IV.
Diagnosis and repair of failed runs

I want to …	Go to
find out why runs failed and repair them, round by round	Section 25.1
read the report and its certainty numbers	Section 25.2.1
set my own parameters for some materials of a run	Section 25.4
check one material or one job quickly	Section 26
try a different INCAR setting on one material first	Section 27
look up a symptom and its remedy	Section 29
lower a high LOBSTER charge spilling	Appendix A.5

25. Repairing failed runs, round by round

After a batch of calculations — one project or a few thousand materials — some runs will not have finished: a job ran out of memory or time, an SCF did not converge, a band structure diverged, LOBSTER stopped. VASP-Flow repairs them in **rounds**. Each round checks every

material, says what went wrong, why, and how sure it is, repairs the inputs of the failed ones *in place* and resubmits exactly what has to be redone. You wait for those jobs and start the next round. One or two rounds are usually enough. A material that fails again with the same cause, or has been repaired five times, is held back for you.

25.1. The repair cycle in one command

Figure 10 shows the whole cycle with its commands.

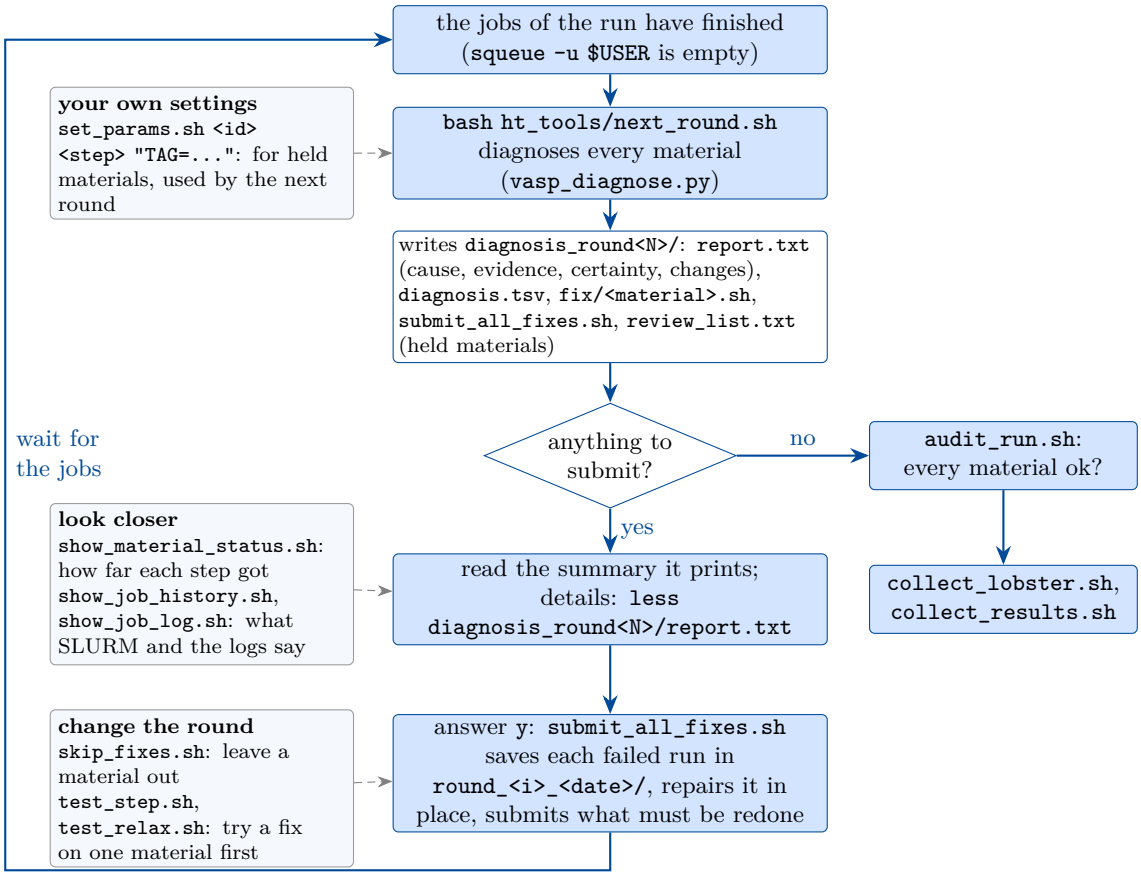


Figure 10: One repair round after another (`ht_tools/` is in the VASP-Flow folder, run from the high-throughput folder). Start at the top when the jobs have finished; stop when `next_round.sh` prints `Nothing to submit.` The diagnosis never changes or submits anything: only the repair scripts do, and only after your `y`. The dashed boxes are optional tools (Sections 25.4, 26, 27 and 28).

Run it from the high-throughput folder (the one with `materials/` and `material_list.txt`) once the queue is empty:

```
cd /scratch/$USER/VASP-Flow/HT_ins_1
bash ~/VASP-Flow/ht_tools/next_round.sh
```

1. It warns if jobs of this run are still in the queue (their materials are skipped this round) and asks whether to go on.
2. It diagnoses every material of `material_list.txt` into a new folder `diagnosis_round1/` (then `diagnosis_round2/`, ...).
3. It prints one line per failed material — the problem, the steps that will be redone, how sure the diagnosis is — and what each repair changes.

4. It asks `Submit the 3 repair(s) now? [y/N]`. With `y` the repairs are submitted in the background (log: `diagnosis_round1/submit_all_fixes.log`).

When those jobs have finished, run `next_round.sh` again. Stop when it prints `Nothing to submit.` Then audit and collect the results (Section 28). `next_round.sh -y` submits without asking, `-n` only diagnoses; other arguments go to `vasp_diagnose.py` (e.g. `--max-rounds 8` to cap the number of repairs, which is not limited by default).

Tip. The diagnosis itself never changes or submits anything. Your runs are touched only by the repair scripts, and only after you answer `y`.

25.2. What a round tells you

The summary counts the materials by problem:

```
Folders checked: 250   (layouts: ht 250)
  ok              245
  spilling_high     2
  stopcar           2
  relax_unconv      1
Fix scripts: 3 to submit, 0 held for review
```

The table below it has one line per failed material: per-step status, the problem and the steps that will be redone, the certainty of the cause and of the fix, and the material's repair round:

```
HT_ins_1__mp-542623   relax:ok scf:stopcar bands:ok lobster:ok
   stopcar -> scf>bands>lobster   cause 0.85  fix 0.90  round 1
HT_ins_1__mp-1080459  relax:relax_unconv scf:ok ...
   relax_unconv -> relax>scf>...   cause 0.95  fix 0.80  round 1
```

The details are in `diagnosis_round<N>/report.txt`, one block per material (held materials first):

```
HT_ins_1__mp-542623   repair round 1 of at most 5
  scf      problem : stopcar   (certainty 0.85)
           cause   : VASP was stopped early by a STOPCAR left behind
              by an earlier run
           seen    : OUTCAR: "hard stop encountered" long before the
              time limit;
                     sacct: job 21298725 COMPLETED after 00:05:12 of
                        01:00:00
  scf      fix certainty 0.90  (rule 0.90; no earlier repairs of
     this cause yet)
  changes : scf: the STOPCAR left by an earlier run is removed; scf
     and the steps after it are redone
  redo    : scf -> bands -> lobster
  backup  : <folder>/round_1_<date>/ (INCAR KPOINTS POSCAR OUTCAR
     OSZICAR EIGENVAL ICO*LIST logs)
  script  : .../diagnosis_round1/fix/HT_ins_1__mp-542623.sh   ->
     submitted by submit_all_fixes.sh
```

`problem` is the name of the cause (the full list is in Section D.5), `cause` says it in words, `seen` is the evidence, and `log` lines, when present, are the error lines from the job logs.

25.2.1. How sure is the diagnosis?

Each block carries two numbers between 0 and 1.

- **Cause certainty** depends on what the diagnosis is based on: SLURM's own record (`TIMEOUT`, `OUT_OF_MEMORY`) or the outputs themselves (OUTCAR says NELM was used up, the relaxation did not converge) 0.95; a known error message in the logs 0.7–0.95 (e.g. `insufficient virtual memory` 0.9, an MPI start-up error 0.7); indirect evidence 0.6; no known message 0.2.
- **Fix certainty** starts from a rule value per cause — how often that repair usually cures it (e.g. doubling the walltime 0.85, halving the LOBSTER mesh after an out-of-memory failure 0.85, `ALGO = All` after NELM 0.7) — and is then corrected by what happened to the earlier repairs of the same cause in the runs checked (from each material's `repair_history.tsv`):

$$c = \frac{4\,p + w}{4 + n},$$

p the rule value, n the earlier repairs of this cause, w of them worked. After a few rounds the number reflects your own runs.

These are rule-based estimates, not the output of a statistical model. Read the blocks with low numbers (below about 0.5) first.

25.3. Where the failed run goes: round backups

Repairs are made in place, in `materials/<id>/`, so the finished steps of a material (e.g. its relaxation and SCF when only LOBSTER failed) are reused and every material always has one folder. Before a repair changes anything, the failed run of each step that will be redone goes into a backup folder of the material:

```
materials/mp-542623/round_1_20261013_0930/
    WHY.txt                  the report block of this repair
    02_scf/  03_bands/  08_lobster/
        INCAR KPOINTS POSCAR (copied: the inputs the failed run used)
        OUTCAR OSZICAR EIGENVAL CONTCAR XDATCAR lobster.out
           ICO*LIST.lobster
        <id>-<id>_<step>-<job>.out/.err        (moved)
materials/mp-542623/repair_history.tsv  one line per repaired step:
   round, date, step, cause, changes
```

i in `round_<i>` counts the repairs of that material (1, 2, ...). The large files that a rerun makes again — WAVECAR, CHGCAR, CHG, vasprun.xml, DOSCAR, PROCAR and the `*CAR.lobster` curves — are deleted, not backed up.

Job scripts stay the same. `job.sbatch` is never edited. A repair changes the step's INCAR (every line of a changed tag is replaced by `TAG = value` at the end) or KPOINTS, and passes the walltime, node count or memory on the `sbatch` command line, which overrides the `#SBATCH` lines of `job.sbatch` for that submission only.

Undo a repair. Cancel the material's jobs, copy the files of `round_<i>_<date>/<step>/` back into `<step>/` and remove the `TAG = value` lines added at the end of the INCAR. `01_relax/POSCAR.orig` is the structure before the first restart.

25.4. Setting parameters yourself

When the report suggests a better setting than the automatic repair — or you just want another one — give it to `set_params.sh` and run the next round:

```
bash ~/VASP-Flow/ht_tools/set_params.sh mp-20352 lobster "KPAR=4; KMESH=10 10 14; TIME=06:00:00"
bash ~/VASP-Flow/ht_tools/set_params.sh mp-542623,mp-972937 scf "NODES=1"
bash ~/VASP-Flow/ht_tools/set_params.sh mp-7889 scf "ALGO=All; NELM=200; SYMPREC="
bash ~/VASP-Flow/ht_tools/next_round.sh
```

Setting	Effect
`TAG=VALUE`	any INCAR tag of that step; `TAG=` (no value) removes it
`KMESH=n1 n2 n3`	the step's Gamma-centred k-mesh
`TIME=HH:MM:SS`	the step's walltime
`NODES=n`	nodes for the step (40 cores each); KPAR is made a divisor of the cores. One node goes to the `standard` partition when the run uses `parallel`

The step is `relax`, `scf`, `bands`, `dos`, `lobster`, or `all` (every step's INCAR; this redoes the whole chain). Several materials are separated by commas. The settings are stored in `materials/<id>/user_params.txt`. In the next round:

- a step whose settings changed after its last run is redone (problem `user_params`), with the steps after it;
- your settings are applied after the automatic repair, so they win;
- they stay in force in later rounds (a repaired step keeps your KPAR, mesh, ...).

`set_params.sh ID -show` lists them, `set_params.sh ID -clear [STEP]` removes them. Nothing is submitted by `set_params.sh` itself.

25.5. Materials held for review

A material's repair script is written but *not* submitted when

- the cause is `unknown`;
- the usual repair of its cause is already in the INCAR (shown as `(fix already in INCAR)`), so it did not help;
- the previous repair of the same step had the same cause (a relaxation that continues from its CONTCAR is an exception: that is progress);
- it has been repaired N times already, when `--max-rounds N` is given (by default the number of rounds is not limited: the two rules above already stop a repair that does not help).

The reason is at the end of its block in `report.txt` (`HELD - ...`), and the material is listed in `review_list.txt`. Then:

- change what you think is wrong with `set_params.sh` and run the next round;
- or accept the result as it is and leave the material out (`bash ~/VASP-Flow/ht_tools/skip_fixes.sh diagnosis_round2 mp-6430`);
- or submit the script as written: in `diagnosis_round<N>/submit_all_fixes.sh` held scripts are commented-out lines at the end — remove the `#` and run it (`--retry-unknown` submits all unknown causes).

25.6. Other runs: projects made with the agents

`next_round.sh` is for high-throughput folders. For anything else — projects of `vasp-agent.py` or `vasp-agent-slurm.py`, several high-throughput runs at once — call `vasp_diagnose.py` with a list of material names or folders; it writes the same files:

```
cd ~/VASP-Flow
python3 vasp_diagnose.py my_list.txt                    # ->
    diagnosis_round<N>/
bash ht_tools/show_problems.sh diagnosis_round1         # what
    failed, what will change
nohup bash diagnosis_round1/submit_all_fixes.sh >
    diagnosis_round1/submit_all_fixes.log 2>&1 &
```

A name is whatever a material's folder is called — `mp-149`, `GaAs`, `GaAs_PBE`; it is looked up next to the list file (Appendix D.2).

Only some runs, or a single project. Make a short list with just those names, or give a folder: `echo GaAs_test > one.txt; python3 vasp_diagnose.py one.txt`.

Runs on another disk. Add `--root` with the folder that holds them, e.g. `--root /scratch/$USER/VASP-Flow`. A run folder holding a `MOVED_TO_SCRATCH.txt` note is followed automatically.

The same name in two runs. All copies are checked and `!! GaAs found in 2 runs` is printed. To check one of them, list its path instead.

Projects with convergence tests. Broken ENCUT/k-point tests are repaired and resubmitted; the production steps are *not* started for you, since you choose their ENCUT and k-mesh from the tests. Such a project shows `phase2_pending` once the tests are complete.

Local projects (no SLURM). Their repair script runs VASP directly; run it where VASP may run (not on a login node): `bash diagnosis_round1/fix/projects__GaAs_test.sh`.

Fewer jobs in the queue; stop submitting. Put `MAX_QUEUED=50` in front of the `nohup bash .../submit_all_fixes.sh` command; `pkill -f submit_all_fixes.sh` stops it (jobs already submitted keep running).

Problems in the log. `awk '/^!!!|^-- /' submit_all_fixes.log` lists failed submissions (`!!!`) and materials skipped because they had jobs in the queue or their outputs changed since the diagnosis (`--`); for the latter, run the round again.

How the diagnosis decides what failed and what to redo — options, the list file, the causes and their repairs, the output files and the limitations — is in Appendix D.

26. Looking up one material or job quickly (`show_*.sh`)

Five short scripts in `ht_tools/` answer the questions that come up while checking runs, so that you do not have to remember long `grep`/`awk` commands. Run them from anywhere (as `bash ~/VASP-Flow/ht_tools/...`; the round folders are relative to where you are). A material can be given by its name (`mp-20352`, `GaAs`) — it is looked up in `/scratch/$USER/VASP-Flow/*/materials/` (and `~/VASP-Flow/*/materials/`, skipping runs

that were moved to scratch) — or as a folder path. A name found in two runs is shown for both. Started without arguments, each script prints how to use it.

Question	Command
Which materials failed, and what will each fix change?	`bash ht_tools/show_problems.sh diagnosis_round3` (`--all`: also the `spilling_high` ones)
Leave some materials out of the resubmission	`bash ht_tools/skip_fixes.sh diagnosis_round3 NAME ...` (`--undo` puts them back)
How far did each step of a material get?	`bash ht_tools/show_material_status.sh mp-20352 [relax scf bands dos lobster]`
What did SLURM do with its jobs?	`bash ht_tools/show_job_history.sh mp-20352 [mp-6430 ...] [-S 2026-10-01]`
What do the job logs say at the end?	`bash ht_tools/show_job_log.sh mp-20352 lobster [LINES] [JOBID]`

Only `skip_fixes.sh` changes a file (it comments lines of `submit_all_fixes.sh` in or out); the others only read.

26.1. A typical checking round

```
cd ~/VASP-Flow
cd /scratch/$USER/VASP-Flow/HT_ins_1
bash ~/VASP-Flow/ht_tools/next_round.sh -n                          #
    diagnose only
bash ~/VASP-Flow/ht_tools/show_material_status.sh mp-20352 lobster #
    a closer look at one material
bash ~/VASP-Flow/ht_tools/show_job_log.sh mp-20352 lobster          #
    ... and at its last log
bash ~/VASP-Flow/ht_tools/skip_fixes.sh diagnosis_round4 mp-6430    #
    do not resubmit this one
nohup bash diagnosis_round4/submit_all_fixes.sh >
    diagnosis_round4/submit_all_fixes.log 2>&1 &
```

26.2. `show_problems.sh`: what failed and what the fixes will do

Prints three blocks: every material of `diagnosis.tsv` that is not `ok` (label, per-step status, planned action, cause and fix certainty, repair round); which fix scripts `submit_all_fixes.sh` will `RUN` and which are `skipped`; and, per fix script, the changes it makes (e.g. `lobster: nscf_diverged -> ALGO = Fast`, `scf: walltime 01:00:00`). Check this before every submission.

26.3. `skip_fixes.sh`: leave materials out

```
bash ht_tools/skip_fixes.sh diagnosis_round4 HT_1-500__mp-6430
    HT_1-500__mp-7889
bash ht_tools/skip_fixes.sh diagnosis_round4 --undo HT_1-500__mp-6430
```

Each name is matched as part of the fix script's name, so `mp-6430` skips the material in every run. It prints the new `RUN`/`skipped` list. Typical uses: a duplicate of a material that another run replaces, or a material whose result you accept as it is (e.g. a relaxation that stops just short of EDIFFG with tiny forces).

26.4. `show_material_status.sh`: how far each step got

For every step folder of the material (or only the steps named):

- a header: `finished` or `not finished`, and **how long VASP ran** — the `Elapsed time` of a finished OUTCAR, or, for an unfinished one, the time from VASP's start to the last write of OUTCAR. This is the real running time, not the walltime limit of the job;
- the INCAR tags that matter for failures (ALGO, KPAR, NCORE, NBANDS, NELM, NSW, EDIFF, EDIFFG, ISYM, ICHARG, ISTART, LDAU);
- **every ionic step** of OSZICAR with the last 5 electronic steps before it;
- for the relaxation: converged or not, the last max force and pressure;
- when VASP did not finish: when OUTCAR was last written and its last 10 lines, which show where VASP stopped (also when it never got to an electronic step).

```
== 01_relax (finished, ran 02:13:26)
   ALGO = Normal
   NSW = 100
   EDIFFG = -0.01
   ...
   DAV:   6     -0.310074960321E+02    -0.42113E-05    ...
   DAV:   7     -0.310074982731E+02    -0.15960E-05    ...
   DAV:   8     -0.310074981222E+02     0.15085E-06    ...
    100 F= -.31007498E+02 E0= -.31007498E+02  d E =0.675983E-02   (8
       electronic steps)
   relax NOT converged
   FORCES: max atom, RMS      0.032426     0.027855
   external pressure =        -0.12 kB  Pullay stress =         0.00 kB
```

Reading OSZICAR. Lines starting with `DAV:` (Davidson), `RMM:` (RMM-DIIS) or `CG:` are *electronic* steps: step number, total energy, its change dE, the change of the band energies, the number of $H\psi$ evaluations and the density residual. A line `N F= ... E0= ... d E = ...` closes *ionic* step N (one relaxation or MD step): `F` is the free energy, `E0` the energy extrapolated to zero smearing, `d E` the change from the previous ionic step (an MD run adds `T=` and `EK=`). An SCF, bands, DOS or LOBSTER run has one ionic line at the end. The status prints the number of electronic steps of each ionic step; if the run stopped in the middle of one it says `... ionic step N still running`.

26.5. `show_job_history.sh` and `show_job_log.sh`: what SLURM and the logs say

`show_job_history.sh` lists, for every job whose name contains one of the patterns, SLURM's record since `-S` (default: 3 days ago): job id, name, state (`COMPLETED`, `TIMEOUT`, `FAILED`, `OUT_OF_MEMORY`, `CANCELLED`), elapsed time, time limit, exit code and end time — and which of these jobs are in the queue now. It shows at a glance whether a step ran out of time, crashed after a minute (an input or memory problem) or was cancelled.

`show_job_log.sh` prints the last lines (default 15) of the newest `.out` and `.err` log of one step, or of the job id given. VASP's own error message is usually in `.out`, the crash trace of MPI in `.err`.

27. Testing a fix on one material first

When it is not clear which setting cures a failure, try a few on one material in short test jobs before changing the production run. Two scripts make test copies in `tests/` of the run folder — outside `materials/`, so the production chain, the diagnosis and the collection scripts never see them.

```
cd /scratch/$USER/VASP-Flow/HT_redo_plainRe
bash ../ht_tools/test_step.sh mp-7889 bands fast    "ALGO=Fast"
bash ../ht_tools/test_step.sh mp-7889 bands kpar4   "KPAR=4; NCORE=2"
bash ../ht_tools/test_step.sh mp-7889 bands ref     ""          #
    unchanged, for comparison
bash ../ht_tools/test_step.sh --report tests/mp-7889_bands_*    #
    while they run, and after
```

Script	What it tests
`test_step.sh ID scf|bands|lobster TAG "T=V; ..."`	The SCF, bands or LOBSTER NSCF of material ID in `tests/ID_STEP_TAG/`: the step's INCAR and KPOINTS, the material's POTCAR and POSCAR and, for bands/LOBSTER, `02_scf/CHGCAR` (no WAVECAR). Only VASP runs, not the LOBSTER program.
`test_relax.sh ID TAG "T=V; ..."`	The relaxation, from the original input cell, in `tests/ID_TAG/`.

Each `TAG=VALUE` replaces that tag in the INCAR (or adds it); `TAG=` with no value removes it (`"NSW=0; IBRION=-1; ISIF="` turns a relax into an SCF). KPAR is lowered to a divisor of the core count if needed. The tests are small and short so that they start soon: 40 cores and 1 hour by default, changed with `T_NTASKS=`, `T_TIME=`, `T_PARTITION=` in front of the command.

`--report` prints one line per test: status (`queued`, `running`, `ended`, `finished`, `timeout`), whether EDIFF was reached, the number of electronic steps, the first, last and largest $|E|$. A largest $|E|$ above $\sim 10^5$ eV means the run diverged. `test_relax.sh --report` gives energy, volume, pressure and max force per ionic step.

Tip. Once a setting works, put it into production with `set_params.sh ID STEP "TAG=VALUE"` and the next round (Section 25.4). Delete `tests/` when done.

28. Auditing, collecting and cleaning up a high-throughput run

When `next_round.sh` has nothing left to submit, check the run once more and pack the results. Run these from the high-throughput folder:

```
cd /scratch/$USER/VASP-Flow/HT_ins_1
bash ~/VASP-Flow/ht_tools/audit_run.sh              # read-only
bash ~/VASP-Flow/ht_tools/collect_lobster.sh        # ->
    ~/lobster_collect_<date>.tar.gz
bash ~/VASP-Flow/ht_tools/cleanup_large.sh          # dry run;
    --apply deletes
```

Script	Purpose
`audit_run.sh [BACKUP ...]`	Read-only check before adding a run to the database: per material missing / queued / relax not converged / SCF not finished or hit NELM / LOBSTER not finished, empty ICOHPLIST or spilling above `SPILL_MAX`; whether it is in a collected archive and in each backup given. Writes `audit_<date>.tsv`.
`collect_lobster.sh`	Tarball of the data the descriptor database needs (raw LOBSTER files, structures, EIGENVALs, trimmed OUTCARs, `_status.tsv`); incremental: a material is packed again only when it changed (e.g. after a repair). `SCF_EIG_ONLY=1` packs only the SCF EIGENVALs of all finished materials.
`cleanup_large.sh`	Delete WAVECAR/CHGCAR no remaining step needs (`02_scf/CHGCAR` is kept unless `--deep`); dry run unless `--apply`.
`fix_spilling.sh [-above 5] [-submit] [ID ...]`	Redo the LOBSTER step of materials whose charge spilling is above the limit: twice the bands, then a LOBSTER basis scan that keeps the lowest spilling (Appendix A.5); a dry run without `--submit`.
`reduce_cell.py IN [OUT]`	The Niggli-reduced cell of a POSCAR (the same crystal on its shortest lattice vectors; numpy only; `modules/cell_reduce.py`). The agents and `ht_runs.py` already do this when LOBSTER runs; use it for runs staged before v2.6.

Full descriptions are in `ht_tools/README.md`.

29. Troubleshooting: symptoms and remedies

29.1. Setting up and running

Symptom	Solution
`No module named 'pymatgen'`, `unexpected keyword argument 'capture_output'`	The `python3` in use is the system one (too old, no pymatgen). Activate the environment of Section 3: `source ~/vaspenv/bin/activate`.
POTCAR not found	Check `VASP_POTCAR_DIR` (LDA: `VASP_POTCAR_DIR_LDA`) in `default_paths` or GUI System Setup. Use the POTCAR chooser for unusual variants (`Ga_d`, `Fe_sv`).
`vasp_std` not found	Set the full binary path in `default_paths` or GUI System Setup.
WARNING: may not have converged	Increase NSW (ionic) or NELM (electronic) in INCAR and re-run.
Wrong bands ($W = \Gamma$)	Conventional cubic cell detected; use the *primitive* cell for FCC/BCC so auto-detection picks cF/cI.
Projected DOS missing	Check instructions.txt: `projected on` and `projections on` are both accepted. Verify element names match POSCAR.
KPAR error	KPAR × NCORE must divide total MPI tasks. Use auto-defaults or set explicitly valid values.
GUI unreachable	Check SSH tunnel: `ssh -L 5001:localhost:5001 user@cluster`.
Project missing from dropdown	Folder must contain `POSCAR` + `settings.json` in the directory where `vasp-gui.py` was started.
Band plot empty or not shown	Usually `sumo-bandplot` crashed on this band structure; the GUI now falls back to a built-in plotter automatically. An empty plot also means a wrong E_F: the Fermi level is read from `04_dos/OUTCAR` (then `02_scf/OUTCAR`), since `03_bands` eigenvalues share the NSCF/DOS energy reference.

29.2. High-throughput runs on a cluster

Symptom	What to do
Jobs stay in the queue as `DependencyNeverSatisfied`	A step before them did not end OK, so they can never start. `scancel` them, then run `next_round.sh` (Section 25); it finds the failed step and writes the repair.
Bands/DOS/LOBSTER energies run away to 10^{6}–10^{10} eV	Pure Davidson diverging at fixed density (seen for Ba_2YReO_6, Sr_2YReO_6). New runs use `ALGO = Fast` in these steps; `vasp_diagnose.py` sets it, then `VeryFast`, for older ones (Appendix A.3).
`forrtl: severe (41): insufficient virtual memory`, the job fails within a minute	Out of memory while setting up. The usual case is the LOBSTER NSCF: with `ISYM = 0` it keeps every k-point of the mesh (6916 for 24×24×24). Halve its mesh in `08_lobster/KPOINTS` (`vasp_diagnose.py` does this; new runs are capped from the start, Appendix A.4). More memory per rank (one node, fewer ranks) did not help. For other steps lower KPAR.
SCF reached EDIFF (`aborting loop because EDIFF is reached` in OUTCAR) but the job still hit its time limit	VASP stalled after convergence (seen on two-node runs, in the MPI network layer), so more walltime will not help. The diagnosis calls it `hang` and reruns the step on one node; `show_material_status.sh ID scf` shows where it stopped. A `1 F=` line alone is no proof: the walltime guard's `LABORT` also ends the electronic loop with it, and such a step (energy change still above EDIFF) was only too slow — a `timeout`, repaired with more walltime (or `set_params.sh ID scf "TIME=04:00:00; NODES=2"`).
A step "finished" in a few minutes, but its SCF is not converged (`hard stop encountered` in OUTCAR)	A `STOPCAR` from an earlier, killed job was still in the folder. The diagnosis calls it `stopcar`, removes it and redoes the step and the ones that used its output. Job scripts made since v2.6 remove a leftover STOPCAR themselves; for older ones, do not resubmit a step by hand without deleting its STOPCAR.
The relax ends `relax_unconv` in every round, with small forces	Check the last max force and pressure (`show_material_status.sh ID relax`). If they are close to EDIFFG and a few kB, accept the structure and leave the material out (`skip_fixes.sh`), or loosen `EDIFFG` with `set_params.sh ID relax "EDIFFG=-0.02"`.
LOBSTER charge spilling above 5 % (`spilling_high`)	The local basis does not represent the bands well. `RWIGS` does not matter to LOBSTER. Run `ht_tools/fix_spilling.sh` (more bands, then a basis scan); if it stays high, see Appendix A.5.
A material shows `queued` in two runs, although only one has jobs	Both runs use the same job name (`<material>_<step>`); diagnose again when the queue is empty.
A run folder in `~/VASP-Flow` looks empty	It was moved to `/scratch/$USER/VASP-Flow` (see its `MOVED_TO_SCRATCH.txt`); work there.

29.3. Quick OUTCAR checks

```
grep "reached required accuracy"   OUTCAR           # converged?
grep "energy  without"             OUTCAR | tail -1 # total energy
grep "E-fermi"                     OUTCAR | tail -1 # Fermi level
```

```
grep "magnetization␣(x)"            OUTCAR | tail -5 # moment
grep "TOTAL-FORCE"                  OUTCAR | tail -20
grep -A3 "BORN␣EFFECTIVE␣CHARGES"   OUTCAR | head -20
```

Appendices: how the inputs are chosen

A. How the inputs are generated

A.1. k-mesh formula

kpra (k-points per reciprocal atom) $= N_x N_y N_z \times N_{\text{atoms}}$. Given a target kpra and the reciprocal lattice vectors $\mathbf{b}_i^*$ (columns of $2\pi(\mathbf{A}^{-1})^\top$), the mesh follows from one formula, with no search:

$$N_i = \text{round}_2(s\,|\mathbf{b}_i^*|), \qquad s = \left(\frac{N_k}{|\mathbf{b}_1^*||\mathbf{b}_2^*||\mathbf{b}_3^*|}\right)^{1/3}, \quad N_k = \text{kpra}/N_{\text{atoms}},$$

where round_2 rounds to the nearest *even* integer (minimum 2), so that BZ boundary points ($k = 1/2$) are on the mesh. Because every axis uses the same s, the spacing $|\mathbf{b}_i^*|/N_i$ is the same along all three axes up to that rounding (a uniform mesh), axes with equal $|\mathbf{b}_i^*|$ get equal N_i, and $N_1 N_2 N_3 N_{\text{atoms}} \approx$ kpra. For hexagonal lattices ($a = b$, $\gamma = 120°$, detected from the POSCAR) the two in-plane N are rounded to multiples of 6 ($6, 12, 18, \ldots$) so that K and M lie on the mesh. For 2-D slabs the same formula is used in the plane (square root instead of cube root) and $N_z = 1$, so the in-plane mesh carries the whole target. The first line of each generated KPOINTS reports the mesh and the kpra actually reached. Any integer may be given instead of `coarse`/`fine` (`KMESH_DENSITY: 3000`); the relaxation has its own density (`RELAX_KMESH_DENSITY`, default coarse).

A.2. KPAR and NCORE rule

When KPAR/NCORE are not specified they are set *per step* from two numbers only: the total MPI rank count N and that step's irreducible k-point count n_k (the spglib reduction of the step's own KPOINTS; the LOBSTER NSCF runs `ISYM=0`, so its full mesh counts).

Quantity	Rule
KPAR	the largest divisor of N that is $\leq n_k$. k-point groups scale almost perfectly, so use as many as there are k-points; a divisor, so no rank is idle.
NCORE	the largest divisor of $r = N/\text{KPAR}$ (ranks per k-group) that is $\leq \sqrt{r}$, VASP's own recommendation (NCORE = 1 for 1–3 ranks per group).

Examples with $N = 40$: $n_k \geq 40 \Rightarrow$ KPAR = 40, NCORE = 1; $n_k = 20 \Rightarrow 20/1$; $n_k = 7 \Rightarrow 5/2$; $n_k = 1 \Rightarrow 1/5$. If only one of the pair is given (instruction key, per-step key or an `INCAR` block), it is kept — snapped down to a divisor of N — and the other is derived from it: a fixed KPAR gets NCORE from the rule above; a fixed NCORE gets the largest KPAR $\leq n_k$ that leaves a multiple of NCORE ranks per group. Priority, highest first: ELF pin (KPAR = 1) > `INCAR` block > per-step key (`SCF_KPAR`, ...) > global `KPAR`/`NCORE` > the rule.

Three steps opt out: **DFPT** is always KPAR = NCORE = 1 (linear response supports neither); **phonons** and **Wannier90** default to KPAR = 1 (independent supercell jobs / the Wannier interface), with NCORE from the rule. With 40 ranks and the dense meshes of a small cell ($\leq$20 atoms, hundreds of irreducible k-points), KPAR = 40, NCORE = 1 is therefore the expected, correct result; it costs one copy of the wavefunctions per rank, which is small for such cells.

Note. `LELF=.TRUE.` requires KPAR = 1. With `ELF: on` (the default) the SCF itself runs LELF and is pinned to KPAR = 1. With `ELF: separate` the SCF keeps the automatic KPAR and a short KPAR = 1 restart writes the ELFCAR afterwards (Section A.8). Convergence tests drop `LELF` automatically.

A.3. Eigensolver (ALGO) of each step

Relaxation and SCF use `ALGO = Normal` (blocked Davidson), the robust choice while the density still changes. The fixed-density steps — bands, DOS and the LOBSTER NSCF (`ICHARG = 11`) — use `ALGO = Fast`: a few Davidson steps, then RMM-DIIS. With the density fixed, pure Davidson ran away to $|E| \sim 10^{10}$ eV for Ba_2YReO_6 and Sr_2YReO_6 (independent of KPAR/NCORE); `ALGO = Fast` converged in 20 steps and `ALGO = VeryFast` (RMM-DIIS only) also converged, while `ALGO = All` (conjugate gradients) is not allowed with `ICHARG = 11` and stops VASP. A functional's own ALGO (HSE06: `Damped`) and any INCAR block win over these defaults. `vasp_diagnose.py` repairs a diverged or unconverged fixed-density step along `Normal` → `Fast` → `VeryFast`, and a non-converging SCF with `ALGO = All`.

A.4. LOBSTER k-mesh and memory

The LOBSTER NSCF must run without symmetry (`ISYM = 0`), so VASP keeps every k-point of the mesh; only time reversal pairs $\mathbf{k}$ with $-\mathbf{k}$:

$$N_\mathrm{k} = \frac{N_1 N_2 N_3 - t}{2} + t, \qquad t = 2^{\#\{i:\ N_i \text{ even}\}}$$

(6916 for 24×24×24, 2820 for 16×16×22, as VASP reports). Its memory grows with $N_\mathrm{k} \times N_\mathrm{PW}$, where the plane waves per k-point follow from the cell volume and the cutoff, $N_\mathrm{PW} = V k_c^3/(6\pi^2)$ with $k_c = \sqrt{\mathrm{ENCUT}/3.81\ \mathrm{eV\ Å^2}}$. Two runs with $N_\mathrm{k} N_\mathrm{PW} = 1.5 \times 10^7$ and 1.8×10^7 stopped at set-up with `insufficient virtual memory`, on two 96-core nodes as well as on one 40-core node with 375 GB — more memory per rank did not help, a smaller mesh does. The generators therefore start from the SCF mesh and, if it has more than `LOBSTER_MAX_KPTS` points ($N_1 N_2 N_3$, default 10000) or $N_\mathrm{k} N_\mathrm{PW} >$ `LOBSTER_MAX_KPW` (default 6×10^6), scale all N_i by the same factor (even, at least 2; $N_z = 1$ for slabs; in-plane multiples of 6 for hexagonal cells) until both hold. The point limit alone would not be enough: 16×16×22 has only 5632 points but failed, because that cell has twice as many plane waves per k-point. The KPOINTS comment then reads e.g. `reduced from 24x24x24: 2052 k x ~2822 PW`. The KPAR rule uses the reduced count. Set either limit to 0 (`LOBSTER_MAX_KPTS: 0`, `--lobster-max-kpw 0`) to switch it off, or a larger `LOBSTER_MAX_KPW` on nodes with more memory. For COHP/ICOHP integrals a mesh of a few hundred to two thousand k-points is ample for cells of up to ∼20 atoms.

A.5. LOBSTER charge spilling: what it is and how to lower it

What LOBSTER projects onto. LOBSTER does not use VASP's projection spheres: `RWIGS` (and `LORBIT`) only change VASP's own site projections (PROCAR, the VASP DOS). LOBSTER projects the PAW wavefunctions in `WAVECAR` onto its own local basis of atomic orbitals: contracted Slater-type orbitals (STOs), $\phi(r) = \sum_i c_i\, r^{n_i - 1} e^{-\zeta_i r}$, fitted to free atoms. The *absolute charge spilling* printed in `lobsterout` is the part of the occupied wavefunctions that this basis cannot represent (averaged over bands and k-points, absolute values since LOBSTER 2). The *total spilling* includes the empty bands and matters much less. LOBSTER's own guidance: below 1 % is very good; about 5 % "could still be considered good", depending on the complexity of the structure; above that, ICOHP/ICOBI values are less reliable. The electrons are not lost — they are relocated in the projection. A small spilling alone does not prove that the basis is complete (LOBSTER FAQ 4.19).

What you can change. The extent and completeness of the orbitals — exactly what determines the spilling — are set by the basis, at three levels:

`lobsterin` keyword	Effect
`basisSet pbeVaspFit2015` *(default)*, `koga`, `bunge`	The tabulated STO sets. `pbeVaspFit2015` is Koga's set plus functions fitted to PBE atoms (Maintz et al., J. Comput. Chem. 37, 1030 (2016)).
`basisFunctions El 3d 4s 4p`	Which orbitals each element gets. Without it LOBSTER uses its *recommended* set, which is the minimal one; optional orbitals — 4p of 3d metals, 5p/6p of 4d/5d metals, p of Li, Na, Mg, Be, 5d of Eu, Yb, Er, 4f/5f of La, Ac — are added only on request. LOBSTER's guide asks you to choose them explicitly, in particular for `_sv` potentials.
`useBasisSet custom` `customSTOforAtom El El.sto`	(LOBSTER $\geq$ 5.0) your own STOs: for each orbital the principal quantum numbers, Slater exponents ζ_i and contraction coefficients c_i of its primitives. This is the only way to change the *radial extent* of an orbital (smaller ζ = more diffuse). LOBSTER ships an example (`Custom_Basis/diamond/C.sto`, format in its `README.md`).

Two more conditions matter. **NBANDS** of the symmetry-off NSCF must be at least the number of basis functions (LOBSTER FAQ 4.11: "match the number of your bands with the number of orbitals"). With NBANDS exactly equal to it, as in VASP-Flow before v2.6, many k-points can fail LOBSTER's orthonormalisation (`WARNING: ... k-points could not be orthonormalized`, `bandOverlaps.lobster`); the guide traces this to "columns that are almost zero in the projected coefficient matrix", i.e. a basis that does not fit the bands. And the **POTCAR** decides which semicore states are valence and must be in the basis.

What VASP-Flow does.

- **NBANDS** of the LOBSTER NSCF is at least the largest pbeVaspFit2015 basis of the cell (min/standard/max per element), times `LOBSTER_NBANDS_FACTOR` (default 1), so a larger basis can always be tried without redoing the NSCF.
- `LOBSTER_BASIS:` in `instructions.txt` chooses the basis: `auto` (default: LOBSTER's recommended set, and a scan when the spilling is above `LOBSTER_SCAN_ABOVE`, default 5 %), `min`, `standard` or `max` (the `basisFunctions` lines are written into `lobsterin`), `scan` (always scan), `off` (never scan).
- The **basis scan** (`08_lobster/lobster_basis.py scan .`) reruns only the LOBSTER program, once per combination of the distinct min/standard/max sets of each element, on the same WAVECAR, and keeps the outputs with the lowest charge spilling; `basis_scan.tsv` lists every try, `basis_scan_best.txt` the choice. A high-throughput job does this automatically before it deletes the WAVECAR (`LOBSTER_SCAN_ABOVE` in `env.sh`). The table of sets, `modules/lobster_basis_table.json`, is pymatgen's `BASIS_PBE_54_{min,standard,max}.yaml`; no pymatgen is needed on the cluster.
- **Runs made before v2.6**: `ht_tools/fix_spilling.sh` finds the materials above `--above` (5 %), backs up their LOBSTER run (`round_<i>_<date>/`), sets NBANDS to `--factor` (2) $\times$ the largest basis, and submits one job: the NSCF, then the basis scan. Without `--submit` it only lists what it would do.

When the scan does not help. If every element has only one tabulated set (Si, C, O, Cs_sv, ...) the scan cannot change the basis, and only more bands remain. If the spilling stays high,

the next steps are by hand: another POTCAR (fewer semicore states, so a smaller minimal basis is adequate), `basisSet koga`, or a custom STO set with more diffuse or additional orbitals. Compare materials of the same chemistry: in the October 2026 run the 2-atom SiC (mp-8062) had 1.5 % with the same basis as the 10-atom polytype mp-11713 (18.3 %), which points to the run rather than the basis.

Skewed cells. For mp-11713 it was the cell: its Materials Project primitive cell has three 12.7 Å vectors 13.9° apart, and the basis scan on it made things worse (58 % at best). The same crystal in its Niggli-reduced cell (3.08, 3.08, 12.71 Å) gave 1.52 % with the default basis. Since v2.6 the agents reduce such a cell before the first step when LOBSTER runs (`REDUCE_CELL`, Section 17); for an older run, reduce the POSCAR with `ht_tools/reduce_cell.py` and run the material again.

What the October 2026 rescan gave. `fix_spilling.sh` on the 52 HT_1-500 materials above 5 %: 46 are now below 4 %, almost all with the larger basis (the semicore p of the transition metal, e.g. Hf–Be 27.3 → 1.6 %, Ti–Se 21.7 → 2.1 %, Zr–Be 21.2 → 1.4 %); Cs_2O (one basis per element) fell from 11.7 to 4.7 % with the doubled bands alone. LiGe (9.9 %) and Re (5.4 %) did not change; for plain Li and Mg LOBSTER printed `-nan` with every basis, so these are rerun with the semicore POTCARs `Li_sv` and `Mg_pv`.

A.6. Relaxation → SCF hand-off

The relaxation starts from scratch (`ISTART = 0`, `ICHARG = 2`) and writes `WAVECAR` and `CHGCAR`. `02_scf/copy_from_relax.sh` copies the relaxed geometry and the CHGCAR; if the SCF k-mesh, spin and cutoff equal the relaxation's it also copies the WAVECAR and the SCF starts from both (`ISTART = 1`, `ICHARG = 1`), otherwise it starts from the CHGCAR alone (`ISTART = 0`, `ICHARG = 1`) because wavefunctions on different k-meshes are not interchangeable. The decision is made at run time, so k-meshes edited or patched after generation are handled. Putting `ISTART` or `ICHARG` in an `INCAR scf:` block disables the mechanism.

A.7. Band-path detection

Since v1.6 the generator first determines the *actual space group* of the POSCAR with spglib (via pymatgen's `KPathSetyawanCurtarolo`) and uses the standard Setyawan–Curtarolo high-symmetry path for that space group. Every high-symmetry point is transformed through Cartesian reciprocal space into the reciprocal basis of *this* POSCAR, so the path is correct for the actual cell used in the SCF regardless of orientation or setting.

If spglib or pymatgen is not installed (or the symmetry analysis fails), the generator falls back to the geometric classifier below: it reads the POSCAR lattice vectors, computes cell lengths and angles, and selects a default path by Bravais type. 2-D structures always use the geometric route so the in-plane path is preserved.

Type	Detection criterion	Default path
cF (FCC)	a=b=c, angles ≈ 60°	G-X-W-K-G-L-U-W
cI (BCC)	a=b=c, angles ≈ 109.5°	G-H-N-G-P-H
cP (cubic)	a=b=c, 90°	G-X-M-G-R-X
hP (hexagonal)	a≈b, γ=120°	G-M-K-G-A-L-H-A
tP (tetragonal)	a≈b≠c, 90°	G-X-M-G-Z-R-A-Z
oP (orthorhombic)	90°, a≠b≠c	G-X-S-Y-G-Z-U-R-T-Z
rP (rhombohedral)	a=b=c, equal angles	G-T-F-G-L
mP (monoclinic)	α=γ=90°, β≠90°	G-Y-A-Z-G-B-D

A.8. Two-step ELF (`ELF: separate`)

`LELF` only works with KPAR = 1, which removes the k-point parallelism that makes an SCF on a small cell fast. With

```
ELF: separate          # or  ELF: two-step
```

the ELF is produced in two steps inside `02_scf`, in the same job:

1. The SCF runs normally — automatic KPAR/NCORE, no `LELF`, `LWAVE/LCHARG=.TRUE.`.
2. `run_elf.sh` (called by `run.sh` after VASP) copies the converged `WAVECAR`, `CHGCAR`, `POSCAR`, `KPOINTS` and `POTCAR` into `02_scf/elf/` and runs VASP there with `02_scf/elf/INCAR`: the SCF INCAR (same ENCUT, smearing, MAGMOM, $+U$, user overrides) plus `ISTART=1`, `ICHARG=1`, `LELF=.TRUE.`, `LWAVE/LCHARG=.FALSE.`, KPAR = 1 and NCORE from the rule for one k-group. `NBANDS` is set at run time to the value the SCF used (read from its `OUTCAR`), so the WAVECAR is read as is. Starting from converged wavefunctions, this restart needs only a few electronic steps.
3. `ELFCAR` is copied back to `02_scf/ELFCAR`, where `makeplots.sh`, `elf_bonds.py` and the GUI look for it; the copied WAVECAR/CHGCAR in `elf/` are deleted.

If the SCF left no WAVECAR/CHGCAR, the ELF pass is skipped with a message. The DOS, bands and LOBSTER steps still read `02_scf/CHGCAR`, which the ELF pass does not touch. `ht-mp-scf.py` uses this mode by default; write `ELF: on` in `instructions.txt` for the old single run.

A.9. Convergence tests: test structure and parameter choice

A.9.1. Shifted test structure

The POSCAR used in `00_convergence` has **atom 1 displaced by** (0.01, 0.02, 0.03) **Å (Cartesian)** so that the force on it is non-zero and its convergence can be tracked — at the equilibrium (high-symmetry) positions the forces would vanish identically and the force panel would be meaningless. The displacement is converted through the lattice for Direct-mode POSCARs. All production steps (`01_relax`, `02_scf`, ...) use the *original, unshifted* positions.

A.9.2. Automatic parameter selection

After the runs, `00_convergence/choose_params.py` (invoked by `run_convergence.sh`, `makeplots_convergence.sh`, and the phase-2 scripts) selects the smallest ENCUT and k-mesh from which every subsequent change stays below

$$\max_i |\Delta F_i^{\text{atom 1}}| < 5 \text{ meV/Å} \quad \text{and} \quad \max_i |\Delta \sigma_{ii}| < 0.5 \text{ kbar},$$

and writes the result to `00_convergence/converged_params.txt`. Phase 2 (`run_calculations.sh`, `submit_calculations.sh`, and the GUI form) offers these values as defaults; when run *non-interactively* — e.g. a convergence test automatically followed by the SCF in a chained/nohup run — they are applied without prompting. If the criteria are not met within the tested range, the largest tested value is used and a `NOT CONVERGED` warning is issued. Thresholds can be overridden: `choose_params.py -ftol 3 -ptol 0.2`.

A.10. DFPT: fixed parallel and symmetry settings

> **Note.** Both routes support neither k-point nor band parallelisation: **KPAR and NCORE are forced to 1** in the generated INCAR (overriding any global or per-step setting). The full MPI task count is still used for plane-wave parallelisation. Likewise **ISYM=2 is enforced** (symmetry imposed): perturbation theory relies on the symmetry-reduced perturbation set, so a symmetry-off `ISYM=0/−1` — e.g. from a global `INCAR:` override block intended for the LOBSTER/NSCF steps — is never inherited by `06_dfpt`.

`copy_from_scf.sh` (invoked by `run.sh`) pulls `WAVECAR` and `CHGCAR` from `02_scf` at run time. If `02_scf` has been converted in place for LOBSTER (`ISYM=0/−1` in its INCAR, as done by the high-throughput `run_scf_for_lobster.sh`), the `WAVECAR` copy is skipped — it holds the full, unsymmetrised k-mesh, which the ISYM=2 run cannot restart from — and VASP regenerates the orbitals from the `CHGCAR`, which the LOBSTER conversion (a fixed-charge NSCF with `LCHARG=.FALSE.`) leaves untouched. If the symmetry-off INCAR is *not* a fixed-charge NSCF, a warning is printed that the `CHGCAR` itself may stem from a symmetry-off SCF and a symmetric SCF should be re-run first.

A.11. LOBSTER projected DOS vs. the VASP DOS

The LOBSTER card also carries a projected-DOS plot (`plots/*_lobster_dos.{png,pdf}`, generated by every interface but displayed only in the GUI), drawn in the same cumulative-stacked layout as the `04_dos` projected DOS. It answers a question the two DOS files raise on their own.

VASP's `LORBIT=11` PDOS projects onto spherical harmonics *inside the PAW augmentation spheres*, which do not fill space, so the interstitial charge is unassigned and $\sum$PDOS $<$ total DOS — severely so for free-electron-like metals (only 25% of the electrons for fcc Al; 72% for $MoSe_2$, whose Mo-*d* and Se-*p* states are compact). LOBSTER instead re-expands the PAW states in a local atomic-orbital basis and performs a Mulliken population analysis, so its decomposition is complete by construction: the stacked areas sum *exactly* to LOBSTER's own total.

The plot therefore shows three things: the stacked LOBSTER channels, LOBSTER's total (dashed grey, identically the top of the stack), and the projection-free VASP total from `04_dos` (solid black). The gap between the last two is the charge spilling, reported in `lobsterout` and typically 1–4%. For the *total* DOS the VASP curve is the reference — it is a direct eigenvalue count under the tetrahedron method, with no projection at all – whereas for *orbital-resolved* weights the LOBSTER decomposition is the more complete one, and is the same basis that produces the COHP/COBI curves. Note that the two curves also carry different broadening (`04_dos` uses `ISMEAR=-5`; the LOBSTER NSCF is forced to Gaussian smearing, and LOBSTER applies its own `gaussianSmearingWidth` on a coarser energy grid), so some of the visible difference is resolution rather than physics. Both smearing settings are stated in the plot legend.

B. How SLURM steps continue past the time limit

A step that reaches its limit is continued, not lost. Every step script asks SLURM for a warning 15 min before the limit (`--signal=B:USR1@900`). On the warning it writes a `STOPCAR` (`LSTOP` for a relaxation: stop after the current ionic step; `LABORT` otherwise), lets VASP finish cleanly and requeues the job with `scontrol requeue` — same job id, so jobs chained after it with `afterok` keep waiting. The requeued job then

- **relaxation**: continues from the `CONTCAR` (or, if that is empty, from the last `XDATCAR` frame, cell included);

- **LOBSTER**: skips the NSCF if it had finished and reruns only the LOBSTER binary;
- other steps: rerun.

The interrupted `OUTCAR`/`XDATCAR` are kept as `*.timeout0`, `*.timeout1`, ... If requeueing is not allowed on the cluster, the job resubmits a copy of itself with the same name and time limit and re-points dependent jobs to it. At most `VF_MAX_RESTARTS` (default 5) continuations per step.

C. High-throughput drivers: all options

C.1. `ht-mp-scf.py` in detail

For each ID it stages `_ht_inputs/<id>/{POSCAR, instructions.txt}` and writes `runall.sh`, which calls `vasp-agent.py` (local) or `vasp-agent-slurm.py` (SLURM) to build the inputs and then runs or submits each material. `runall.sh` downloads nothing.

Relaxation first. `01_relax` relaxes cell shape, volume and ions (`IBRION = 1`, `ISIF = 3`), because the MP cell was relaxed with a different functional/setup. It uses the SCF k-point density (`--relax-kpra` to change it), so the SCF starts from the relaxed `CONTCAR`, `CHGCAR` and `WAVECAR` (`copy_from_relax.sh`).

ELF in two steps. The instructions carry `ELF: separate`: the SCF runs with the automatic KPAR/NCORE, then a short KPAR = 1 restart in `02_scf/elf/` writes `ELFCAR`, copied to `02_scf/ELFCAR` (Section A.8).

k-points. Uniform mesh at `--kpra` (default `coarse` = 1000); per material via `KMESH_DENSITY` in `_ht_inputs/<id>/instructions.txt`; `--kmesh` forces one fixed grid for all (not recommended for mixed cell shapes).

Option	Meaning
`-l`, `--list`	ID file (default `highthrouput_list` in the current directory).
`--agent`	`local` or `slurm`; prompts if omitted.
`--poscar-dir DIR`	Reuse downloaded POSCARs (see above).
`--chain` / `--no-chain`	SLURM: chain every step of every material with `afterok` (default), or submit each material independently (its own relax → SCF → LOBSTER chain). Local runs are always sequential.
`--relax` / `--no-relax`	Relaxation first (default on).
`--relax-kpra`	k-density of the relaxation (default: `--kpra`).
`--gga_u auto|on|off`	GGA+U flag (default `auto`: only chalcogenides/halides).
`--kpra`	`coarse`, `fine` (6000) or an integer.
`--kmesh`	Fixed Γ-mesh, e.g. `'8 8 8'`; overrides `--kpra`.
`--encut`	ENCUT (eV); default 500.
`--functional`	PBE (default here, as in the existing database), R2SCAN, PBEsol, LDA, ...
`--lobster` / `--no-lobster`	LOBSTER step (default on).
`--kpar`, `--ncore`	Force KPAR/NCORE (give one, the other is derived); the ELF pass always uses KPAR = 1.
`--mpi`	MPI ranks per job (workstation).
`--walltime`	One walltime for every step (default: per-step defaults, Section 21.3).
`--nodes`, `--ntasks-per-node`, `--partition`, `--account`, `--profile`	SLURM resources, written into each `instructions.txt` (they override `profiles/slurm.json`).

Output	Location
Material folders	`<mp-id>/01_relax, 02_scf, 08_lobster, plots`
ELFCAR	`<mp-id>/02_scf/ELFCAR` (ELF pass in `02_scf/elf/`)
Staged inputs	`_ht_inputs/<mp-id>/`
Element / Z table	`_ht_inputs/elements_Z.txt`

Note. A download error `'dict' object has no attribute 'number'` is an spglib/pymatgen version mismatch: `pip install -U spglib`. The driver falls back to `Structure.get_primitive_structure()` automatically.

C.2. `ht_runs.py` in detail

It builds a self-contained `highthroughput_*` directory: one `materials/<id>/job.sbatch` per material; `submit_all.sh` submits it once per step (`relax`, `scf`, `bands`, `lobster`) as an `afterok` chain with the walltimes `TIME_RELAX/TIME_SCF/TIME_BANDS/TIME_LOBSTER` of `env.sh`, and steps that reach their limit continue automatically (Section 21.3). The steps are the relaxation (`01_relax`, IBRION = 1, ISIF = 3, same k-density as the SCF; if the first pass does not reach the force criterion it is restarted once from its CONTCAR), the SCF from the relaxed structure, bands and the LOBSTER NSCF + LOBSTER (finished steps are skipped on resubmission); the LOBSTER job ends and ends with `makeplots.sh` (plots and `lobster_summary.csv` in `plots/`). ENCUT and the LOBSTER NBANDS come from the POTCARs at generation time and are checked against `potcar_manifest.json` when the POTCARs are built on the cluster. POTCAR variants follow the Materials Project choices by default (`--potcar-set mp`: e.g. Re_pv, Ti_pv, Ga_d, Li_sv, taken from pymatgen's `MPRelaxSet`); the older "plain element first" rule (`--potcar-set minimal`) gave minimal-valence POTCARs, and Ba_2YReO_6/Sr_2YReO_6 with plain Re collapsed into an unphysical state. Relax, SCF and bands use the same `NBANDS` = 1.2× VASP's default (from NELECT, NIONS and the starting moment; rounded up to even): VASP's default leaves too few empty bands for narrow d bands, the electron count then drifts between SCF steps (`BRMIX: very serious problems`) and the SCF diverges. The bands step never uses fewer bands than its SCF. The k-point density is `--kpra` (default 8000 k-points per reciprocal atom), the same spacing along all axes. The LOBSTER NSCF uses the SCF mesh, reduced when it exceeds `--lobster-max-kpts` or `--lobster-max-kpw` (Appendix A.4). Every job requests the whole node's memory (`#SBATCH --mem=0`). `--kspacing` is an alternative to `--kpra`; before v2.5 it was the default (0.10 $Å^{-1}$, very dense: 24×24×24 for the 8-atom $Sc_2Al_2C_2O_2$, 110592 kpra). 3d elements (V–Ni) start spin-polarised. GGA+U follows the default rule (only chalcogenides/halides, Section 12.4) unless `--gga_u on|off` is given. There is no DFPT step (dielectric tensors and Born charges are ill-defined at zero gap, and the linear-response loop tends not to converge) and no ELF (`ELF: off`).

Option	Meaning
`--list`, `--out`	ID file and directory to create.
`--kpra N`	k-points per reciprocal atom (default 8000; 1000 = coarse).
`--kspacing D`	instead of `--kpra`: fixed spacing, $N_i = \text{even}\lceil \lvert\mathbf{b}_i^*\rvert / D \rceil$ (0.10 Å^{-1} is very dense).
`--lobster-max-kpts N`	largest $N_1 N_2 N_3$ of the LOBSTER mesh (default 10000, 0 = no cap).
`--lobster-max-kpw X`	LOBSTER memory budget, k-points × plane waves (default 6×10^6, 0 = no cap).
`--relax` / `--no-relax`	Relaxation first (default on).
`--no-reduce-cell`	keep a skewed cell as downloaded (default: Niggli-reduced at staging, original in `POSCAR.input`).
`--gga_u auto|on|off`	GGA+U flag written into every `instructions.txt` (default `auto`: only chalcogenides/halides).
`--potcar-set mp|minimal`	POTCAR variants: Materials Project choices (default) or plain element first, then _sv/_pv/_d.
`--single-node`	Every material on one node with `--cores-per-node` (default 40) ranks; otherwise large cells get more nodes.
`--kpar`, `--ncore`	Force KPAR/NCORE (default automatic).
`--max-atoms`, `--max-nodes`	Skip larger cells / node limit.
`--build-potcars` / `--no-build-potcars`	Build POTCARs right after generation (default: when `sbatch` exists).
`--include-potcars`	Ship assembled POTCARs in the directory.
`--poscar-dir DIR`	Reuse downloaded POSCARs (see above).
`--partition`, `--account`, `--modules`, `--jobs N`	Prepare N materials in parallel (default min(8, CPUs)).
`--vasp-std`, `--lobster-bin`, `--hpc-potcar-dir`	Cluster settings, written into `env.sh` and the job scripts.

D. How `vasp_diagnose.py` works

D.1. Command line

```
python3 vasp_diagnose.py LIST [options]
```

Option	Default	Meaning
`--out DIR`	`diagnosis_round<N>`	folder for the report and the scripts; $N = 1 +$ the last round in the current folder
`--root DIR`	—	another folder to look up material names in (can be repeated)
`--since DATE`	14 days ago	how far back SLURM's job records (`sacct`) are read, YYYY-MM-DD; must reach back to the jobs being diagnosed
`--kpar-max N`	8	largest KPAR written by the out-of-memory repair
`--spill-max X`	5	LOBSTER charge spilling (%) above which a material is flagged
`--max-rounds N`	0	repairs of one material before it is held for review (0 = no limit)
`--retry-unknown`	off	also submit the materials whose failure was not recognised
`--max-queued N`	200	default queue limit of `submit_all_fixes.sh` (override later with `MAX_QUEUED=`)

It needs only Python ≥ 3.6 (no extra packages) and uses `sacct` and `squeue` when they exist.

D.2. The list file

One entry per line; empty lines, `#` comments and anything after the first word are ignored. An entry is

Entry	Checked
a **material name** — any folder name: `mp-149`, `GaAs`, `Bi2Se3_soc`, ...	the folders of that name in R, R`/*/` or R`/*/materials/`, where R is the folder holding the list file, and each `--root`
a **material or project folder** (has `00_convergence`, `01_relax`, ...)	that folder
a **run folder** with `materials/`	every material in `materials/`; if the run has a `material_list.txt`, only the materials in it (other folders are named and skipped)
any **other folder**	every sub-folder that is a material/project folder

A word that is a folder (relative to the list file's folder) is taken as a path; otherwise it is a material name. Names are matched exactly, so `GaAs` finds `.../GaAs` but not `.../GaAs_PBE`,

and a name may contain any characters a folder name can. A name that is found nowhere is reported as `missing`.

D.3. Kinds of runs

How the jobs are resubmitted depends on how the run was made:

Layout	Recognised by	Resubmitted with
`ht`	`job.sbatch` in the material folder (high-throughput drivers)	`sbatch job.sbatch <step>`, walltimes from `TIME_*` in the run's `env.sh`; steps relax, scf, bands, lobster
`slurm`	`run.sh` files with `#SBATCH` lines (`vasp-agent-slurm.py`)	`sbatch <step>/run.sh`, with its job name and walltime
`local`	anything else (`vasp-agent.py`)	`(cd <step> && bash run.sh)`, in the foreground

Each material gets a *label*, `<run folder>__<material>` (e.g. `HT_ins_1__mp-149`, `projects__GaAs_test`); it names the repair script and the report line.

D.4. When is a step OK?

Step	OK when
`00_convergence`	every point of the ENCUT test (`encut/encut_<E>/`) and of the k-point test (`kpoints/<N1>x<N2>x<N3>/`) that its `run.sh` will make has finished with a converged SCF. If `converged_params.txt` says a quantity did not converge in the tested range, this is noted in the report (widen the range yourself)
`01_relax`	OUTCAR has `reached required accuracy`
`02_scf`	OUTCAR finished (`General timing and accounting`) and the SCF converged (`aborting loop because EDIFF is reached`)
`03_bands`	finished, converged, and no energy above 10^6 eV in OSZICAR
`04_dos`	as bands, and a non-empty DOSCAR
`06_dfpt`	finished with Born charges or the dielectric tensor in OUTCAR
`08_lobster`	`lobster.out` has `finished in` and `ICOHPLIST.lobster` is not empty; a charge spilling above `--spill-max` is reported but counts as OK

`05_wannier` and `07_phonons` are not checked (noted as `unchecked`).

For a step that is not OK, the cause is read from the step's newest SLURM logs, `vasp.out`, the end of OUTCAR and the SLURM job record (state, run time, time limit). The job record wins: a job SLURM ended for its time limit is a `timeout`, whatever else is in the logs — or a `hang`, if the step is an SCF or fixed-density run whose electronic loop had reached EDIFF (`aborting loop because EDIFF is reached` in OUTCAR, and its `1 F=` line in OSZICAR) but which never reached the final timing block. An `F=` line without EDIFF comes from the walltime guard's stop, and is a `timeout`. A VASP run stopped by the STOPCAR that the VASP-Flow scripts write 15 minutes before the time limit (`hard stop encountered`, `soft stop encountered`) is a `timeout` as well; stopped that way long before its time limit (or with a STOPCAR still in the folder and no requeue pending), it is a `stopcar`. Messages after which VASP carries on, such as `Sub-Space-Matrix is not hermitian` or `The number of bands has changed`, are shown in the script header but never taken as the cause.

D.5. Failure causes and their repairs

Name	Meaning	Repair
`oom`	out of memory (`insufficient virtual memory`, `Cannot allocate memory`, killed by the OOM handler, MPI time-outs between nodes)	KPAR lowered to at most `--kpar-max` (a divisor of the MPI ranks); all node memory requested; for the LOBSTER NSCF also its k-mesh halved per axis
`timeout`	the job hit its time limit	walltime doubled; a relax continues from its last structure
`hang`	an SCF/fixed-density run converged, then stalled until the time limit	rerun on one node, with the other redone static steps; on one node already: walltime doubled
`stopcar`	VASP stopped early by a STOPCAR an earlier, killed job left behind	STOPCAR removed; the step and the steps that used its output redone
`scf_nelm`	SCF stopped at NELM without converging	`ALGO = All`, `NELM` ≥ 150
`nscf_diverged`	bands/DOS/LOBSTER energies exploded	30% more bands than the SCF (not for LOBSTER); eigensolver `Normal` (Davidson, VASP's default) $\rightarrow$ `Fast` $\rightarrow$ `VeryFast` (RMM-DIIS), then held; `NELM` ≥ 150
`nscf_unconv`	bands/DOS stopped at NELM	`ALGO` as for nscf_diverged, `NELM` ≥ 150
`relax_unconv`	relax ended without reaching the force criterion	continues from CONTCAR (or the last XDATCAR frame)
`zbrent`	`ZBRENT: fatal error` in a relax	`IBRION` 2 $\rightarrow$ 1, continue
`symmetry`	VASP symmetry errors	`SYMPREC = 1E-4` (`ISYM = 0` if SYMPREC was set)
`edddav`	`EDDDAV`/`ZHEGV` errors	SCF: `ALGO = All`; relax: `ALGO = Normal`, if that fails again `ALGO = All`; bands etc.: as nscf_diverged
`wavecar`	WAVECAR does not fit the new run	WAVECAR moved aside, start from CHGCAR
`chgcar`	CHGCAR unreadable or cut off	SCF started from scratch (`ICHARG = 2`)
`chgcar_missing`	a later step needs `02_scf/CHGCAR`, which is missing or cut off	SCF redone first
`lobster_fail`	VASP part of the LOBSTER step finished, the LOBSTER program did not	only the LOBSTER program is rerun
`quota`	disk full / quota exceeded	resubmitted unchanged — **free space first**
`mpi_init`	MPI failed to start (bad node)	resubmitted unchanged
`not_run`	never ran (e.g. an earlier step failed)	submitted
`user_params`	you changed the step's settings with `set_params.sh` after its last run	redone with them (Section 25.4)
`unknown`	none of the above	script written but held back for review
(any) `... (fix already in INCAR)`	the repair for this cause is already in the INCAR, so it did not help	script written but held back for review
(any)`, twice`	the previous repair of the step had the same cause, or `--max-rounds N` repairs done (if given)	script written but held back for review
`spilling_high`	LOBSTER finished, spilling > `--spill-max`	not by the round; run `ht_tools/fix_spilling.sh` (Appendix A.5)
`missing`	a listed name has no folder	none (report only)

A material is counted under the cause of its *first* failed step. A charge spilling is not changed by rerunning the same step; lowering it needs more bands and other basis functions (`fix_spilling.sh`, Appendix A.5).

D.6. What is redone

- If the relax or the SCF failed, that step and *all* later steps are redone.
- Otherwise only the failed later steps (bands, DOS, DFPT, LOBSTER). If they need `02_scf/CHGCAR` and it is missing or cut off, the SCF is redone first.
- **Convergence tests** are repaired on their own: only the broken points are moved aside, the fix of their cause goes into the test's INCAR, and the test's `run.sh` is resubmitted — it skips the points that are already done. Production steps are not redone because of a convergence test.
- **LOBSTER only**: when the VASP part of the LOBSTER step finished and its WAVECAR is there, only the LOBSTER program reruns (not for local projects).
- In high-throughput runs `job.sbatch` cannot run DOS or DFPT; such steps are reported (`job.sbatch cannot run: dos`) but not resubmitted.

What a repair script changes in a material folder:

- **INCAR**: every line of a changed tag is removed and `TAG = value` is added at the end. No other tags are touched.
- **The failed run** of every redone step goes to `<folder>/round_<i>_<date>/<step>/` first (Section 25.3): inputs copied, small outputs and job logs moved, WAVECAR/CHGCAR/vasprun.xml and the other large files deleted; a broken convergence point is moved there as a whole. Older repairs (before v2.6) used `stale_<date>/`; both count as repair rounds.
- **Your settings** from `user_params.txt` are applied after the automatic changes.
- **Nodes**: a step that hung on several nodes, and the other static steps redone with it, get `-nodes=1`; a step whose last run used fewer nodes than `job.sbatch` asks for keeps that number. KPAR is lowered to a divisor of the new number of ranks.
- **Relax restarts** from the newest complete CONTCAR (else the last XDATCAR frame); the original POSCAR is kept once as `POSCAR.orig`.
- **SCF start**: if the relax's CHGCAR is cut off, the SCF starts from scratch.

D.7. Order of the resubmitted jobs

Layout	Order
`ht`	each step waits for the previous one to succeed (`afterok`); LOBSTER after bands waits only for it to end (`afterany`), as it does not need the bands
`slurm`	SCF waits for the relax; bands, DOS, DFPT and LOBSTER wait for the SCF
`local`	one after the other; stops at the first failure

Convergence tests are submitted without waiting for anything. Walltimes are the run's own (`env.sh`, `#SBATCH --time`, or relax 8 h, SCF 4 h, bands/DOS 2 h, DFPT 8 h, LOBSTER 4 h, tests 8 h), doubled after a time-out.

D.8. Output files

diagnosis.tsv — tab-separated, one line per material:

Column	Content
label	the material's label
layout	ht, slurm, local (- for a missing name)
path	its folder
steps	step:status for each step (conv:encut, conv:kpoints, relax, ...) and notes
action	ok, a report-only name, or <cause> -> <steps redone>; REVIEW in front when held back
script	its repair script, if any
round	the material's repair round (i of round_<i>)
cause_certainty, fix_certainty	as in report.txt (Section 25.2.1); the fix value is the lowest over the material's broken steps

```
awk -F'\t' 'NR>1␣{split($5,a,"␣");␣n[a[1]]++}␣END␣{for␣(k␣in␣n)␣
    print␣k,␣n[k]}' diagnosis.tsv
awk -F'\t' '$5␣~␣/^REVIEW/␣{print␣$1}' diagnosis.tsv
```

report.txt — one block per failed material (Section 25), held ones first; failed_list.txt lists the folders that got a repair script, review_list.txt the held ones with the reason.

fix/<label>.sh — a stand-alone bash script; it can be run on its own. In order it:

1. records the diagnosis and the error lines in its header;
2. **stops** if the material has jobs in the queue, or if any of its OUTCARs or lobster.out changed since the diagnosis (so it is safe to run late, or twice);
3. writes the round backup and its WHY.txt, then applies the changes;
4. submits the jobs (or runs them, for a local project) and adds a line per repaired step to repair_history.tsv;
5. leaves a marker .submitted/<label> in the output folder.

submit_all_fixes.sh — runs every repair script in turn; skips those with a .submitted marker; waits while you have MAX_QUEUED or more jobs in the queue (all your jobs count); lists the held-back unknown scripts at the end as commented-out lines.

D.9. Limitations

- Jobs are recognised by their job names (<material>_<step>, or the name in the step's run.sh). Jobs submitted under other names are not seen, and a failure cause is then taken from the log files only.
- Only jobs since --since are known to the tool. Without a job record a time-out is still recognised from the log, but the walltime is doubled from the configured value.
- A second out-of-memory failure of the same step is held for review (same cause twice); lower KPAR or the mesh with set_params.sh, or diagnose with a smaller --kpar-max.

- The certainty numbers are rule-based estimates (Section 25.2.1), corrected only by the repair history of the runs checked.
- `quota` and `mpi_init` are resubmitted unchanged: free disk space first; node failures are usually transient.
- `05_wannier` and `07_phonons` are not checked; a convergence-test range that was too small is reported but not widened.
- Two runs holding the same material use the same job names (`<material>_<step>`), so a job of one makes the material show as `queued` in both.
- Repair scripts use GNU `sed` and `stat`: they are meant for Linux clusters.

E. Version history

v1.0 (2026-03-23) Initial release — GUI, workstation agent, PBEsol/R2SCAN/HSE06, SOC, GGA+U, convergence tests, Wannier90, phonopy.

v1.1 (2026-04–05) Per-task KPAR/NCORE smart defaults; SLURM agent with dependency-chained submission; cumulative DOS by element (no pymatgen); copy-from-relax/scf scripts.

v1.2 (2026-06) Collinear magnetism: MAGMOM per atom, spin-resolved band and DOS plots. Auto k-path detection (8 crystal systems). Fixed DOS projection parsing. Separate GUI and agent quick-reference sheets. Rewritten comprehensive LaTeX manual.

v1.3 (2026-06-15) Raw INCAR passthrough (`INCAR: ... END_INCAR`, global or per-step) for arbitrary VASP tags. Constant-pressure relaxation via `PRESSURE` (ISIF=3, IBRION=2, PSTRESS).

v1.4 (2026-06-19) High-throughput driver `ht-mp-scf.py`: Materials Project primitive cells → batch SCF + ELF, with per-material element/Z table and cross-material `afterok` chaining on SLURM.

v1.5 (2026-06-24) Robust band plotting across the GUI and both CLI agents (workstation + SLURM): the built-in `vasprun.xml` plotter now handles non-spin as well as spin, and is used automatically when `sumo-bandplot` crashes or is unavailable. Fixed the matplotlib fallback (`plot_results.py`) to resolve the real Fermi level and emit PNG, not just PDF.

v1.6 (2026-07-08) Band k-path derived from the real space group via spglib (Setyawan–Curtarolo) in all agents and the GUI, with the geometric classifier kept as fallback. New `08_lobster` step: LOBSTER COHP/COBI/COOP bonding analysis with GUI plots and the `lobster_postprocess.py` aggregator; the NSCF uses ISYM=0, or ISYM=−1 when SOC is on. Full GPL-3.0 text in `LICENSE` (GitHub now detects the license correctly). Manual and quick-reference sheets brought up to date.

v1.7 (2026-07-08) DFPT step overhaul: KPAR/NCORE forced to 1 (linear response and PEAD support neither); automatic finite-field route (`LCALCEPS`, IBRION=6) when SOC, HSE06 or R2SCAN is requested; `extract_born.py` now reports ε_∞, the ionic contribution, total static ε_0, an acoustic-sum-rule check, element-labelled Z^*, and clamped-/relaxed-ion piezoelectric tensors.

v1.8 (2026-07-09) ELF bond profiles: `elf_bonds.py` plots the electron localization function along non-equivalent nearest-neighbour bonds (atom centre to atom centre), wired into `analyze.sh` for both CLI agents and shown in the GUI Results tab whenever `02_scf/ELFCAR` exists. Output is named after the reduced formula (`MoSe2_elf_bonds.pdf`). Detects the all-zero `ELFCAR` that VASP writes for non-collinear / spin-orbit runs and reports it instead of emitting a blank plot.

v1.9 (2026-07-22) *K-mesh and GGA+U.* Physics-based cell-aware k-mesh generation replacing fixed uniform grids. Two density tiers: **coarse** (1000 k-points per reciprocal atom, kpra) for convergence tests and relaxations; **fine** (5000 kpra) for SCF, DOS, and spectroscopic steps. Subdivisions $N_i \propto |\mathbf{b}_i^*|$ from reciprocal lattice vectors, all-even to include BZ boundary points; $N_z = 1$ for 2-D slabs. Hexagonal systems: fixed 6×6 (coarse) or 12×12 (fine) in-plane, N_z from $|\mathbf{b}_3^*|/|\mathbf{b}_1^*|$. DOS mesh always $2N_x \times 2N_y \times 2N_z$. DFPT forced to match SCF mesh exactly. New instruction key `KMESH_DENSITY` (coarse/fine). GGA+U anion gate extended to all chalcogenides and halides (O, F, S, Se, Te, Cl, Br, I); Hubbard lookup table now covers sulfides such as $AgCrP_2S_6$.

Plot improvements. ELF bond-profile and LOBSTER (COHP/COBI/COOP) plot legends and statistics boxes moved outside the axes area so they never obscure curves. Shared plotting module `modules/cohp_plot.py` used by both the GUI and the CLI agents; COHP, COBI, and COOP plots are now generated automatically in `analyze.sh` for both the workstation and SLURM agents, matching GUI behaviour.

Structure visualizer. New *Visualize* button in the POSCAR card: converts the current POSCAR to XYZ via `modules/poscar2xyz.py` and opens it in Jmol, VESTA, or any other viewer. The viewer path is set under System Setup → *Structure visualizer* (or `VISUALIZER_X` in `default_paths`).

Configuration. `LOBSTER_X` and `VISUALIZER_X` are now exposed in the GUI System Setup page and propagated to the agent subprocess environment. Platform config file renamed from `site.env` to `default_paths` for clarity; three-layer precedence (default_paths < GUI settings < profile JSON) documented. Auxiliary Python scripts (`band_plot.py`, `dos_plot.py`, `elf_bonds.py`, `lobster_postprocess.py`, `conv_choose.py`, `poscar2xyz.py`) consolidated into `modules/`.

v2.0 (2026-09-28) *Input-consistency release.* `MAGMOM:` now honours negative values, per-element (`Fe=4.0, O=0.6`) and per-atom lists, switches ISPIN on by itself, and initialises SOC runs along the requested x/y/z direction. K-mesh density is reflected in every KPOINTS file: coarse/fine/integer kpra, even $N_i \propto |\mathbf{b}_i^*|$ chosen for uniform spacing and closeness to the target, hexagonal in-plane meshes in multiples of 6, `RELAX_KMESH_DENSITY`; one generator routine (`kpoints_text`) serves both agents and the GUI. KPAR/NCORE: ELF-pinned KPAR=1 re-derives NCORE, and values given in INCAR blocks steer the companion value. Relaxation starts from scratch and writes WAVECAR/CHGCAR; the SCF reads CHGCAR, and WAVECAR too when the k-mesh matches. Explicit `INCAR <step>:` and new `KPOINTS <step>:` blocks override everything (GUI: Setup → Advanced). New `check_agent_consistency.py` compares GUI, workstation and SLURM outputs. New `ht-semimetals.py` builds the `highthroughput_*` SCF/bands/LOBSTER screening sets from list files.

v2.1 (2026-09-30) *Simpler k-mesh and parallel rules; two-step ELF.* k-mesh: one formula, N_i = nearest even (hexagonal in-plane: multiple of 6) of $s|\mathbf{b}_i^*|$, replacing the tolerance search. KPAR = largest divisor of the ranks $\leq n_k$; NCORE = largest divisor of the ranks per k-group $\leq$ its square root. Wannier90 and phonons now really default to KPAR = 1, and LOBSTER uses its own (ISYM=0) k-count. New `ELF: separate`: SCF at full KPAR,

then a short KPAR = 1 restart writes ELFCAR (default in `ht-mp-scf.py`, which also gains `--kpra` (default coarse), `--kpar`, `--ncore`). `ht-semimetals.py`: `--kpar`/`--ncore`, POTCARs built automatically.

v2.2 (2026-09-30) *Relax-first high throughput; results viewer.* `ht-mp-scf.py`: `01_relax` (IBRION = 2, ISIF = 3) before the SCF by default (`--no-relax`, `--relax-kpra`); SLURM chaining now submits every step of a material, not only the SCF. `ht-semimetals.py`: DFPT step removed; each job runs `analyze.sh` at the end (`ANALYSIS_PYTHON` in `env.sh`), plus `analyze_all.sh`; `collect_results.sh` keeps the per-step layout. GUI: **Open results folder** views jobs run elsewhere and draws missing plots; SCF/NSCF steps show as done when VASP finished. NumPy 2 fix (`np.trapz` → `np.trapezoid`) in the COHP plot and LOBSTER post-processing. GGA+U flag `GGA_U: ON|OFF` in `instructions.txt` (GUI: auto/on/off; HT drivers: `--gga_u`); without it the tabulated U_{eff} is used only for chalcogenides/halides (O, S, Se, Te, F, Cl, Br, I), otherwise $U = 0$; same U in all steps; U values only from `hubbard_u_defaults.csv` (3d/4d/5d/4f/5f, `LDAUU` = U_{eff}, `LDAUJ` = 0). SLURM: walltime per step (relax 8 h, SCF 4 h, bands/DOS 2 h, LOBSTER 4 h); a step reaching its limit is stopped cleanly and requeued (relaxations continue from CONTCAR/XDATCAR); `ht-semimetals.py` submits one job per step. `ht-semimetals.py` relaxes cell + ions first (`01_relax`, ISIF = 3).

v2.3 (2026-10-05) `ht-semimetals.py`: Materials Project POTCAR choices by default (`--potcar-set mp`); `NBANDS` = 1.2× VASP default in relax/SCF/bands, bands never below the SCF; `#SBATCH --mem=0`; relaxation with IBRION = 1. Maintenance tools in `ht_tools/` (`fix_failed.sh`, `test_relax.sh`, `collect_lobster.sh`, `reset_stale.sh`, `cleanup_large.sh`).

v2.4 (2026-10-12) *Checking and repairing runs.* New `vasp_diagnose.py`: checks the runs of a `*_list.txt` (any material or folder names; `00_convergence` to `08_lobster`; high-throughput, SLURM and local layouts), names the cause of each failure, writes a repair and resubmission script per material and `submit_all_fixes.sh`. Bands, DOS and the LOBSTER NSCF use `ALGO = Fast` (pure Davidson diverged at fixed density for Ba_2YReO_6/Sr_2YReO_6). `ht_tools/`: `test_step.sh` (test SCF/bands/LOBSTER settings on one material), `show_problems.sh`, `skip_fixes.sh`, `show_material_status.sh`, `show_job_history.sh`, `show_job_log.sh`. Manual reorganised by task (getting started, single calculations, high-throughput screens, checking and repairing runs; how things work in the appendices), with the `vasp_diagnose.py` manual merged in.

v2.5 (2026-10-12) k-meshes: `fine` = 6000 kpra (was 5000); `ht-semimetals.py` uses `--kpra 8000` by default (`--kspacing` only on request). The LOBSTER NSCF uses the SCF mesh (was 2×), kept within limits by default: with `ISYM = 0` the mesh is reduced (same ratios) above 10000 points (`LOBSTER_MAX_KPTS`) or when k-points × plane waves exceed `LOBSTER_MAX_KPW` (6×10^6); `vasp_diagnose.py` halves the LOBSTER mesh after an out-of-memory failure. `test_step.sh`: one-node tests go to the `standard` partition.

v2.6 (2026-10-13) Repairs in rounds: `ht_tools/next_round.sh` diagnoses a high-throughput run into `diagnosis_round<N>/`, shows the failures and submits the repairs after a y/N question. `report.txt` explains each failure (cause in words, evidence) with a certainty for the cause and for the fix (rule value corrected by the run's repair history); `failed_list.txt`, `review_list.txt`. The failed run is backed up in `<material>/round_<i>_<date>/` (inputs, small outputs, logs, `WHY.txt`; large files deleted) and recorded in `repair_history.tsv`; held for review after the same cause twice or 5 repairs. `ht_tools/set_params.sh`: your own INCAR tags, k-mesh, walltime and node count per material and step, applied by the next round. New causes

`hang` (reached EDIFF, then stalled on several nodes: rerun on one node; a step the walltime guard stopped before EDIFF is a `timeout`) and `stopcar` (stopped early by a leftover STOPCAR); job scripts remove a leftover STOPCAR at the start. `ht-semimetals.py` renamed `ht_runs.py`. Removed the superseded `fix_failed.sh`, `reset_stale.sh`, `resume_in_scratch.sh`, `redo_lobster.sh`, `patch_*.sh`. LOBSTER charge spilling: NBANDS of the LOBSTER NSCF covers the largest pbeVaspFit2015 basis; `LOBSTER_BASIS` (auto/min/standard/max/scan/off) and an automatic basis scan above `LOBSTER_SCAN_ABOVE` (5 %) that keeps the lowest spilling (`lobster_basis.py`); `ht_tools/fix_spilling.sh` for older runs; `ht_tools/reduce_cell.py` (shortest lattice vectors of a skewed cell, without pymatgen). R2SCAN is the default functional of the GUI, `vasp-agent.py` and `vasp-agent-slurm.py` (PBE, PBEsol, LDA and the others on request; the high-throughput drivers keep `--functional PBE`); R2SCAN INCARs carry `LMIXTAU = .TRUE.` for the fixed-density steps, and its automatic Hubbard U is 0.6 times the tabulated PBE value. LDA now sets `GGA = CA` and uses the LDA POTCARs (`VASP_POTCAR_DIR_LDA`, default: the PBE folder with PBE → LDA); before, it ran with the PBE POTCARs, i.e. as PBE. `check_setup.py` / `make check`: is this computer ready (Python and packages, POTCARs, VASP/MPI, `profiles/slurm.json`, high-throughput packages)? Manual reorganised for new users: what the tools are for (preparing and submitting, diagnosis and repair, analysis, above all in high-throughput runs), a route chooser, requirements per use and one installation procedure (the Python environment with pymatgen for the high-throughput drivers), one table of where each setting comes from, a task card per example; then five step-by-step examples (GUI, workstation, cluster, high throughput on a workstation and on a cluster with repair rounds), then how each part works and how to change it by hand; convergence tests before the k-mesh and the steps they set, and separate sections for LOBSTER, the dielectric constant and Born charges, Wannier90 and phonons; parts renamed (Getting started, Worked examples, Reference, Diagnosis and repair); Example 1 is also the GUI reference, with screenshots of the Setup and Results tabs (`figures/make_gui_figures.py`); folder drawings of one calculation and of both high-throughput drivers in Examples 2, 4 and 5; a flowchart of the repair rounds; `make check` described after the installation steps. SOC without magnetism: `Methods: SOC` (GUI: *SOC, non-magnetic*) writes `MAGMOM = 3N*0` instead of leaving VASP's default of 1 μ_B per atom along (1, 1, 1); "non-magnetic" in the instructions no longer switches magnetism on; the MoS_2 and Bi_2Se_3 examples are non-magnetic. The plotting script `analyze.sh` is now `makeplots.sh` and writes to `plots/` (was `analysis/`); likewise `makeplots_all.sh`, `makeplots_convergence.sh` and `plots/makeplots.log`. The GUI, `collect_lobster.sh` and `fix_spilling.sh` still read older runs. GUI defaults from the POSCAR: the *K-path* field is filled with the spglib path and *Hexagonal BZ* is ticked only for a hexagonal cell (was: always ticked, G-M-K-G); `|` starts a new k-path segment; hand-typed labels take spglib's coordinates for the cell; the GUI's LOBSTER help says the SCF mesh (not twice it). Relaxation plots: total energy per ionic step and, for a volume relaxation, the pressure (`modules/relax_plot.py`; `makeplots.sh` of both agents and a *Relaxation* card on the GUI Results tab). A skewed cell is Niggli-reduced before the first step when LOBSTER runs (`modules/cell_reduce.py`; `REDUCE_CELL: auto|on|off`, GUI checkbox with a note of both cells, `ht_runs.py --no-reduce-cell`; original kept as `POSCAR.input`): the 15R-SiC primitive cell went from 18 % to 1.5 % charge spilling. `vasp_diagnose.py` no longer limits the number of repair rounds (`--max-rounds N` sets a limit).